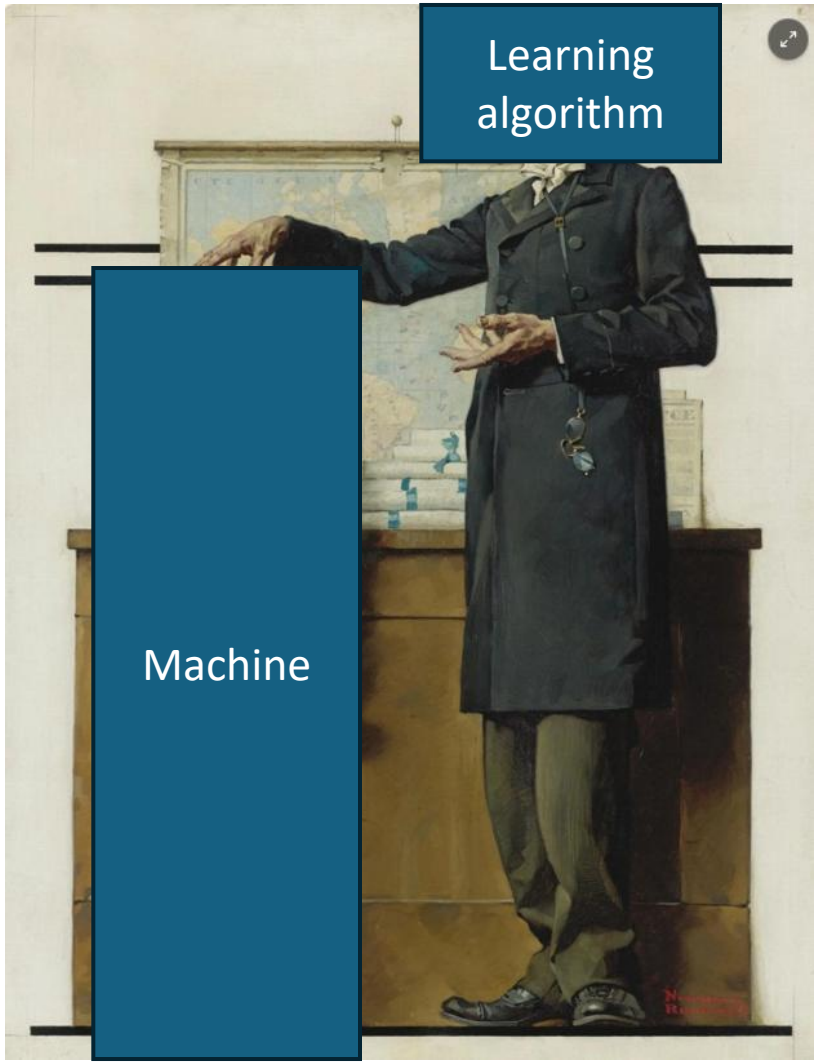
Learning

Norman Rockwell, Graduation

During learning we change the way we solve a task

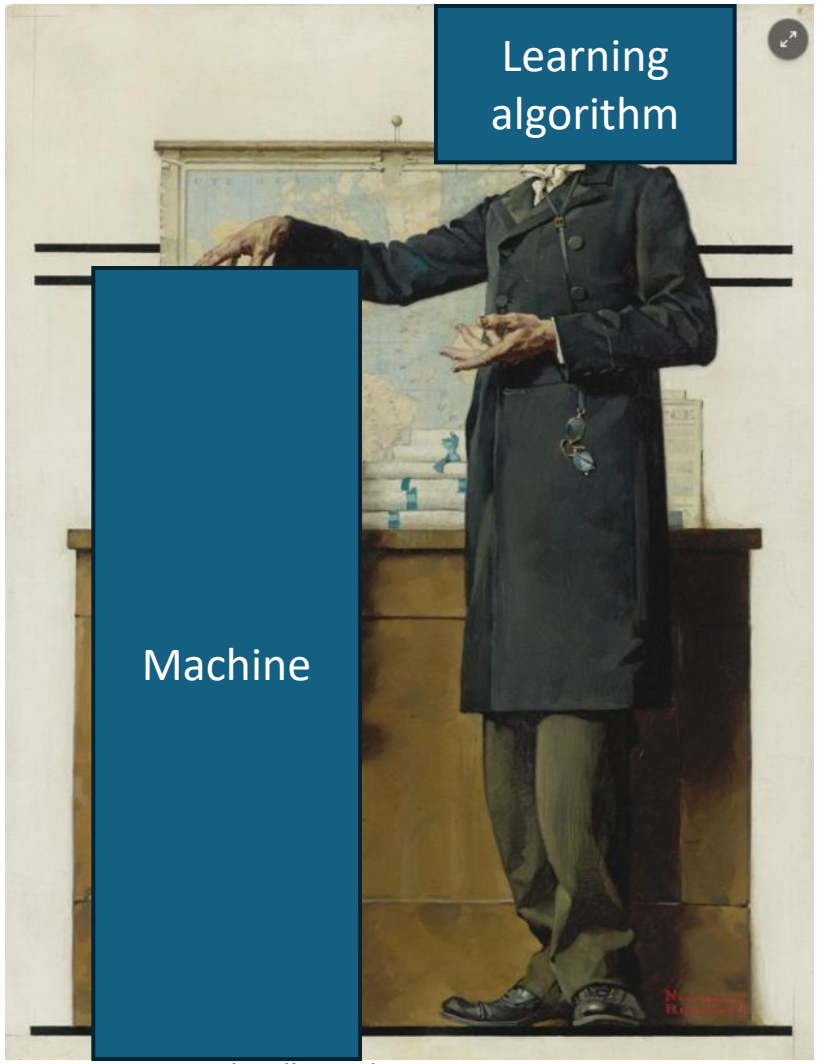
Machine Learning

Norman Rockwell, Graduation



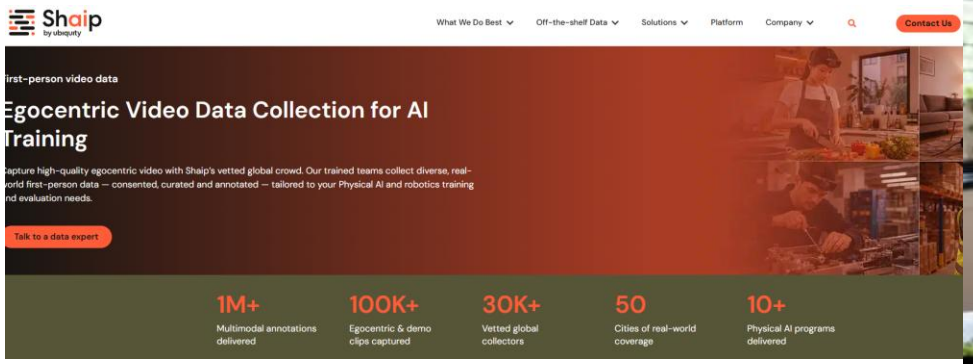
In machine learning we design machines that can change the way they solve a task, and design algorithms that aim to do this optimally

Machine Learning

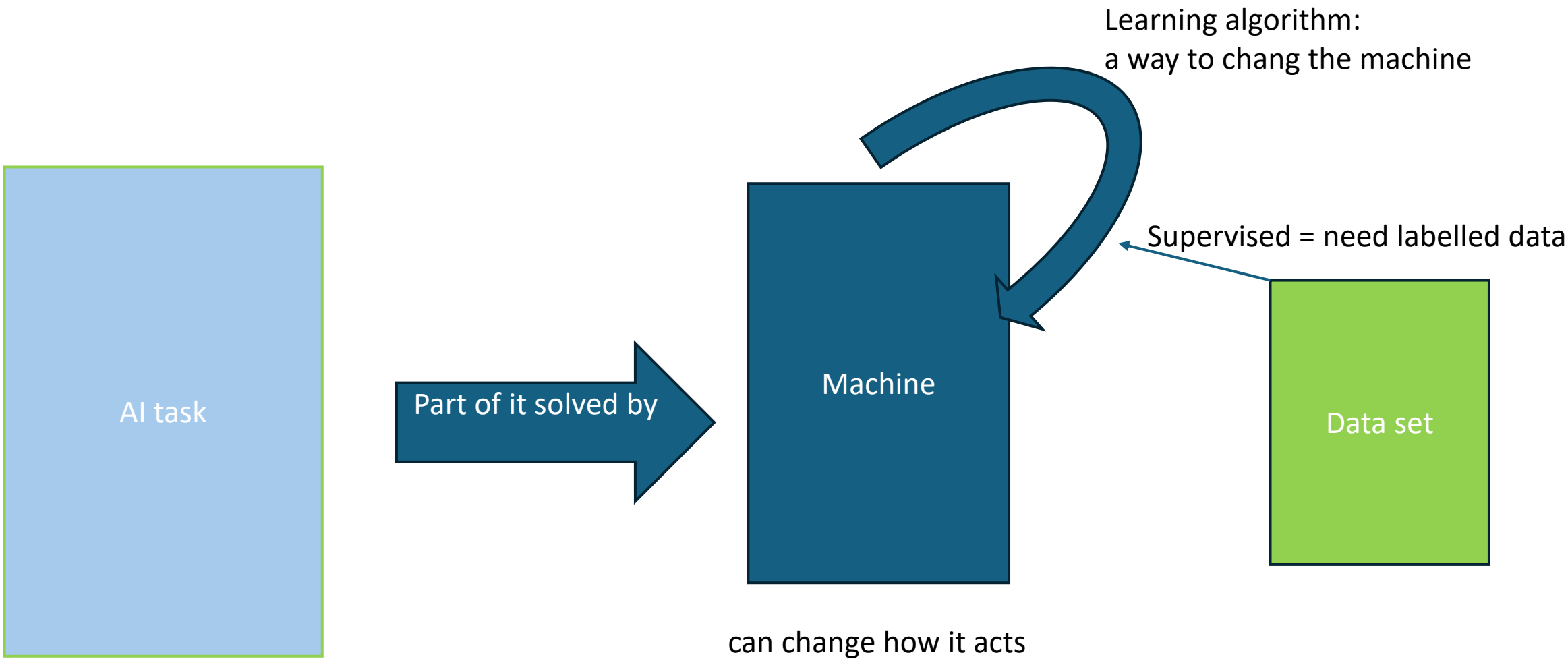


Norman Rockwell, Graduation

In supervised machine learning we use huge data sets of questions and expected answers in our learning algorithms.



Supervised Machine Learning



A task in need for an **optimal** solution

Supervised Machine Learning

Input: List of input output pairs

( , 4) , ( , 8) , ...

Result: A map f from input space to output space

f() = 3

No guarantee that it is correct!

Supervised Machine Learning

ML algorithms are programs that use the labelled examples to chang an initial mapping f_0 so that f_{final} is more correct

1. Iteration of the ML program

$f_0($  $) = 3$

$f_0($  $) = 9$

Change= learning

2. Iteration of the ML program

$f_1($  $) = 5$

$f_1($  $) = 8$

N. Iteration of the ML program

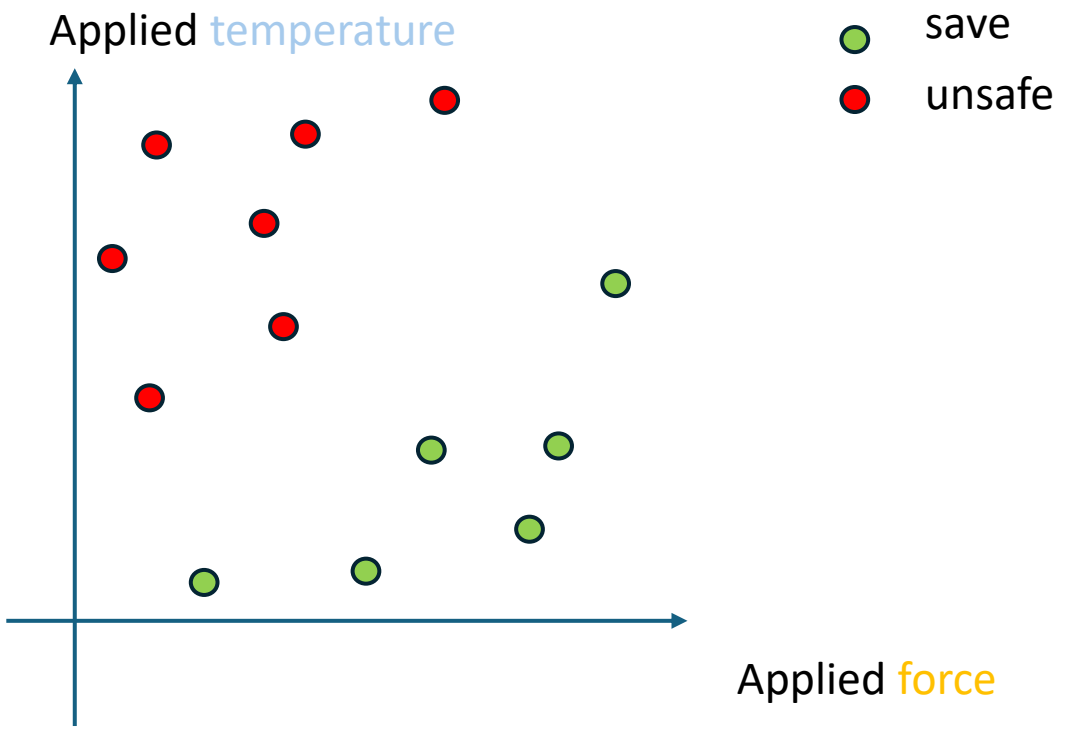
$f_N($  $) = 4$

$f_N($  $) = 8$

ML studies what kind of functions can be used and how they can be updated

Supervised Machine Learning

Simpler example: Is it save to use a given tool in an environment with certain maximal forces and temperatures



Input: ((temp ,force), save)

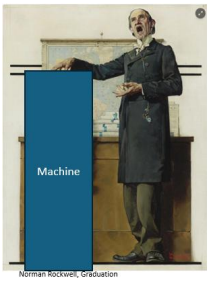
Q: How many colours do I need for the MNIST example? How many dimensions has the input?

Supervised Machine Learning

In order to solve the simple example with ML we need a function that we can update

This is also called the architecture

$$f_W(x) = \begin{cases} 1, x_1 W_1 + x_2 W_2 > 0 \\ 0 \quad , \quad else \end{cases}$$



Each choice of W defines a mapping.
Update means to change W.

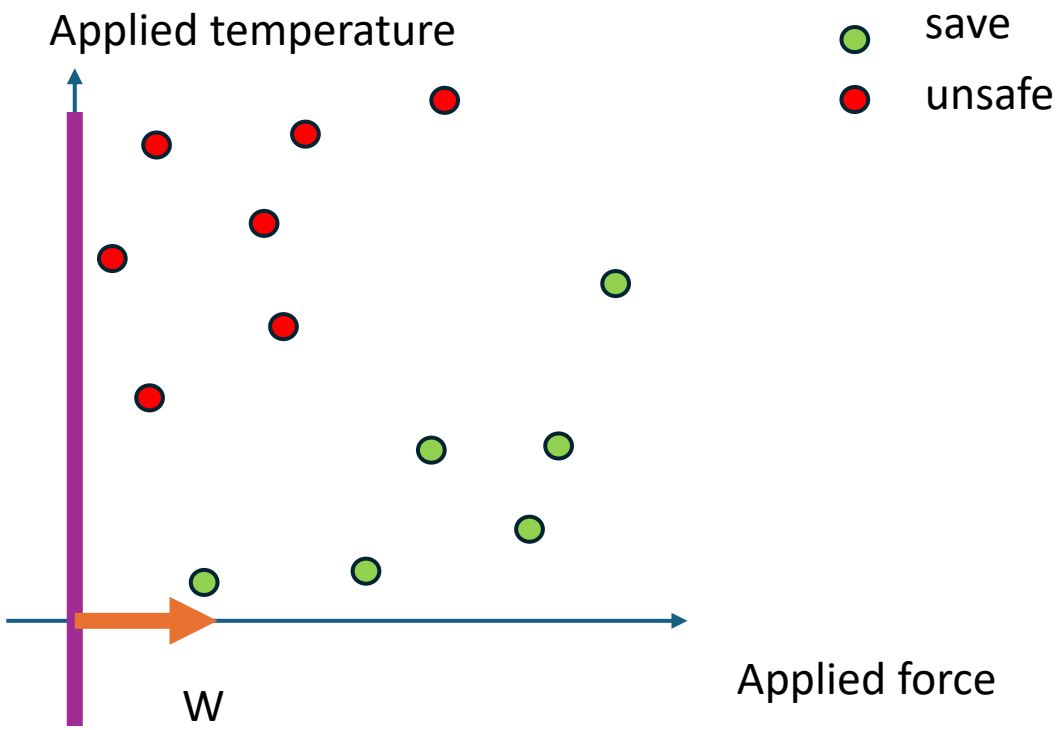
Q: How would you visualize the predictions for the one where W=(1,0)?

A: I colour each point according to its predicted label.
B: A circle around the origin, what's outside of the circle gets the value 1.
C: A line through the origin, the line separates the examples that get the value 1 from the ones who get the value 0.

Supervised Machine Learning

Simpler example: Is it save to use a given tool in an environment with certain maximal forces and temperatures

$$f_W(x) = \begin{cases} 1, x_1 W_1 + x_2 W_2 > 0 \\ 0 \quad , \quad else \end{cases}$$



W=(1,0)

Q: Which inputs will get label 1?

A: the ones on the purple line;

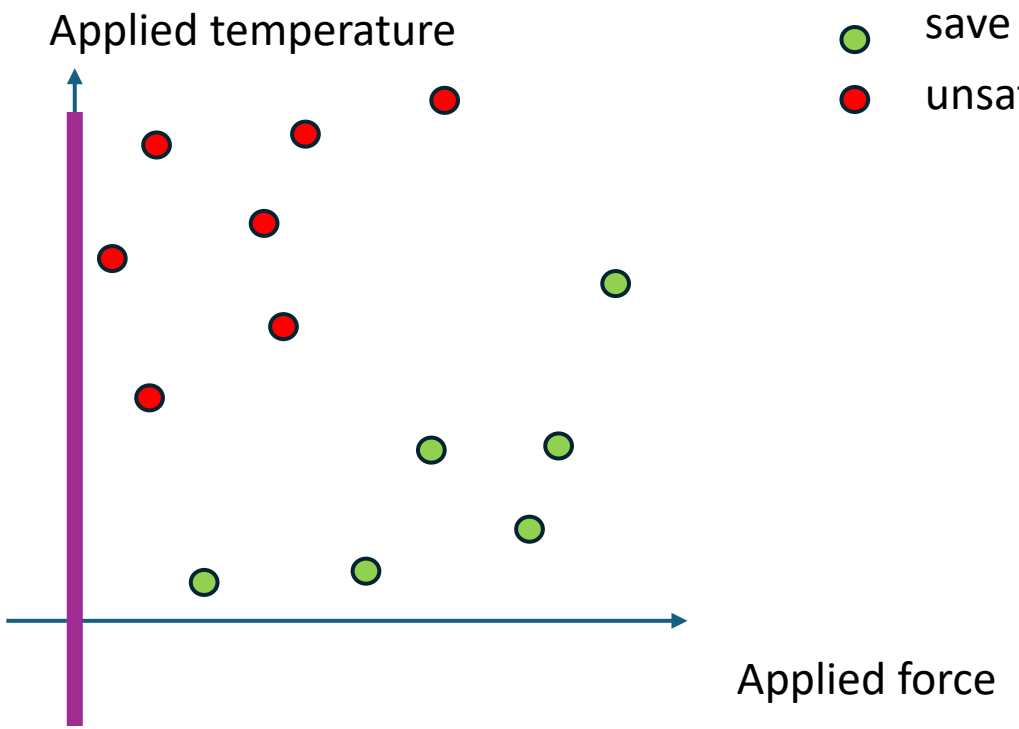
B: the ones that are on the side of the purple line where W is pointing to.

C: the ones that are on the opposite side of the purple line where W is pointing to.

Supervised Machine Learning

Simpler example: Is it save to use a given tool in an environment with certain maximal forces and temperatures

$$f_W(x) = \begin{cases} 1, x_1 W_1 + x_2 W_2 > 0 \\ 0 \quad , \quad else \end{cases}$$



W=(1,0)

Which inputs will get label 1?

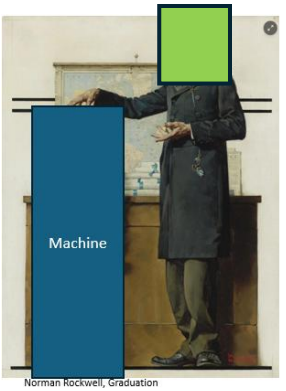
Q: If I choose **W=(100,0),** which input will get label 1?

A: the purple line rotates.

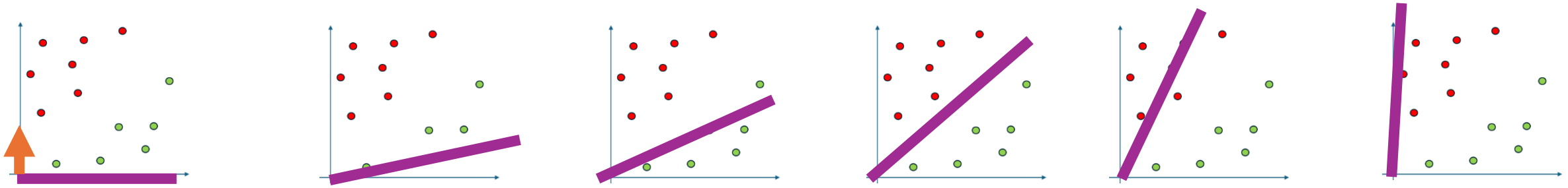
B: the purple line moves to the right.

C: the purple line stays the same.

Supervised Machine Learning



Simpler example: Simple Algorithm
1: replace W by W=$(\cos(\theta), \sin(\theta))$
2: divide the interval [0,360] in 100 pieces
3: return the angel where the prediction and the true label coincide most of the time!



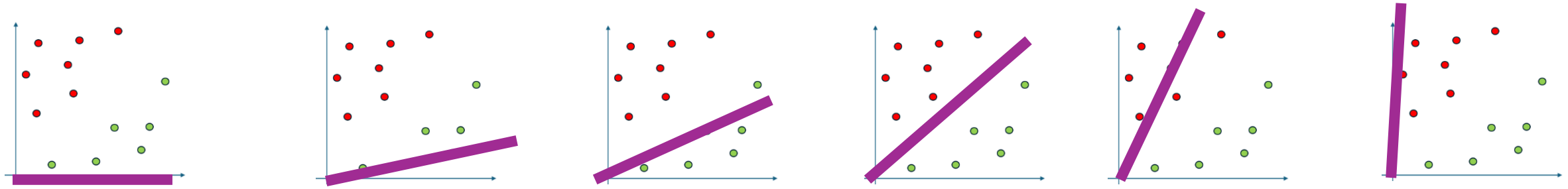
1 2 3 4 5 6

Which one is the best?

Supervised Machine Learning

Simpler example: Simple Algorithm
1: replace W by W=$(\cos(\theta), \sin(\theta))$
2: divide the interval [0,360] in 100 pieces
3: return the angel where the prediction and the true label coincide most of the time!

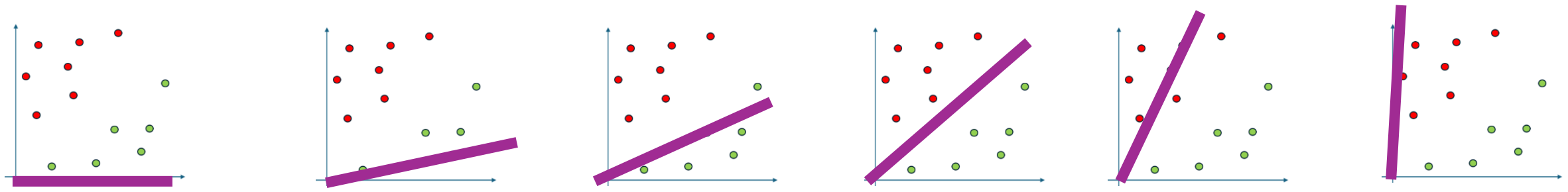
Q: Is 180 enough?



1 2 3 4 5 6

Supervised Machine Learning

Simpler example: Simple Algorithm
1: replace W by W=$(\cos(\theta), \sin(\theta))$
2: divide the interval [0,360] in 100 pieces
3: return the angel where the prediction and the true label coincide most of the time!

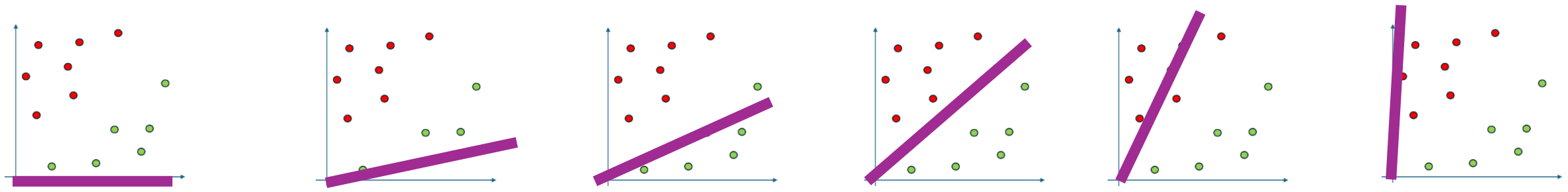


1 2 3 4 5 6

We use a lot of the problem structure in particular that we have a 2D problem. Assume we have a 1000 dim problems
1: is there a singel theta? A: Yes B:No
2: how close will we be to the true line, if we still consider 100 lines? A: As close, B: not as close, C: Even closer

Supervised Machine Learning

Simpler example: Simple Algorithm
1: replace W by W=$(\cos(\theta), \sin(\theta))$
2: divide the interval [0,360] in 100 pieces
3: return the angel where the prediction and the true label coincide most of the time!



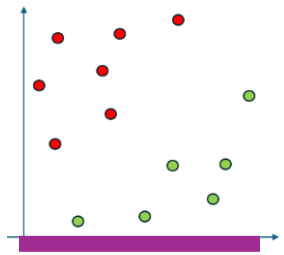
1 2 3 4 5 6

This kind of abstraction
is what makes AI science valuable

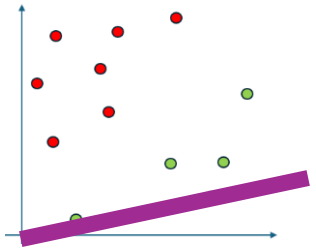
Stay in 2D: Will this simple algorithm work for any 2D classification problem?

Supervised Machine Learning

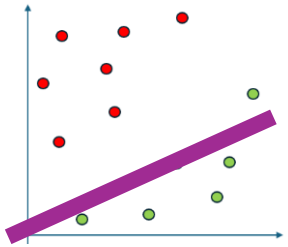
Simpler example: Simple Algorithm
1: replace W by W=$(\cos(\theta), \sin(\theta))$
2: divide the interval [0,360] in 100 pieces
3: return the angel where the prediction and the true label coincide most of the time!



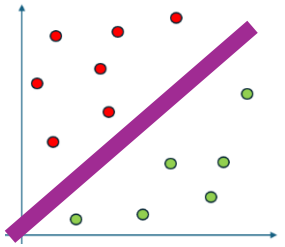
1



2

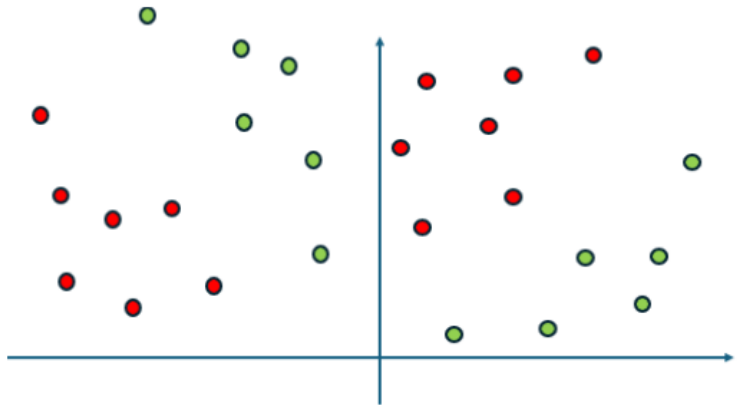


3



4

Stay in 2D: Will this simple algorithm work for any 2D classification problem?



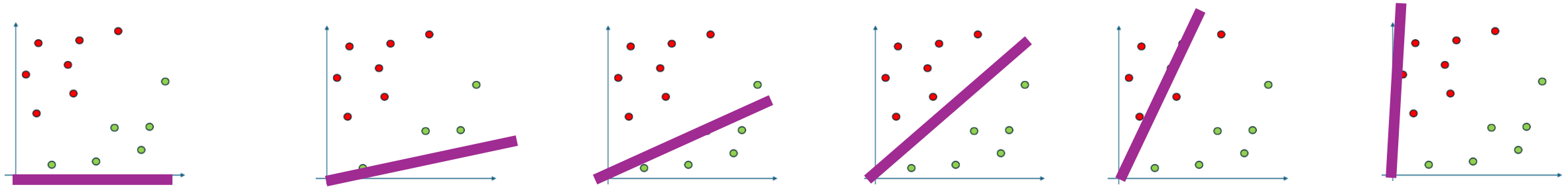
We deal with this later

Supervised Machine Learning

This (2&3) is called gird search

Simpler example: Simple Algorithm
1: replace W by W=$(\cos(\theta), \sin(\theta))$
2: divide the interval [0,360] in 100 pieces
3: return the angel where the prediction and the true label coincide most of the time!

This implicitly defines a loss.



1 2 3 4 5 6

Supervised Machine Learning

We look for an architecture and a loss that can be optimized with quicker methods then grid search.

Lets play a game:
1. I choose a number m in the interval [-0.5,0.5] a number N<100 and weights $s_i \geq 0$ for i=0..N.
2. You tell me a number x in [-1,1]
3. I tell you the result of

$$f_s(x) = \sum_{i=0}^{N} s_i(x-m)^{2*i} = s_0 + s_1(x-m)^2 + s_2(x-m)^4 + \cdots$$

and I tell you the gradient at x.

You must determine m (unto an error of 1/1000) with as few repetitions of step 2&3 as possible
(I must keep all my choices constant)

How would grid search work?
Use the fact that $f_s(x) = 0$ if and only if $x = m$.
If none of the gird points has value zero, why is the x with the smallest value the one closest to m?
Do you see an disadvantage in fixing the grid before seeing any of the function values?

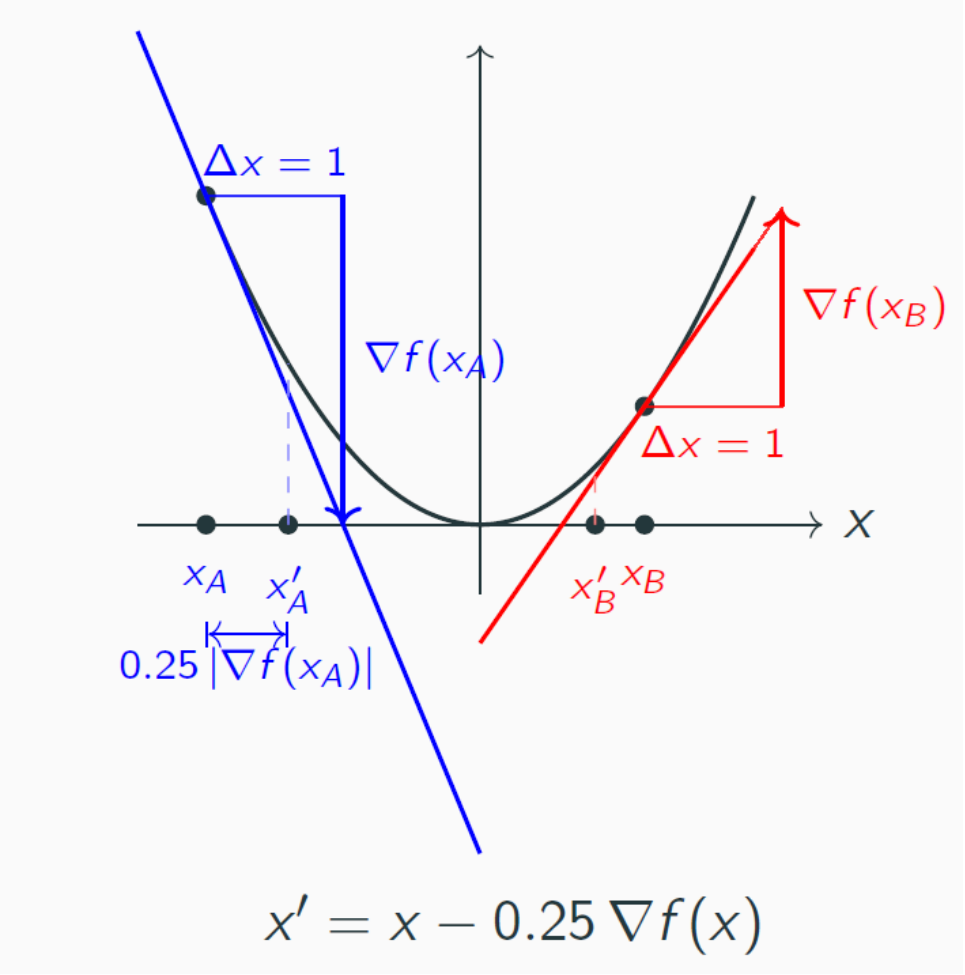
Supervised Machine Learning

Gradient descent

1. Start at some x, for example x=0.
2. Determine the gradient g(x) and update x to x-μg.
3. If μ is small enough then f(x-μg)<f(x). (Is true for all differential f. Why?)

$$f_s(x) = \sum_{i=0}^{N} s_i (x - m)^{2*i}$$

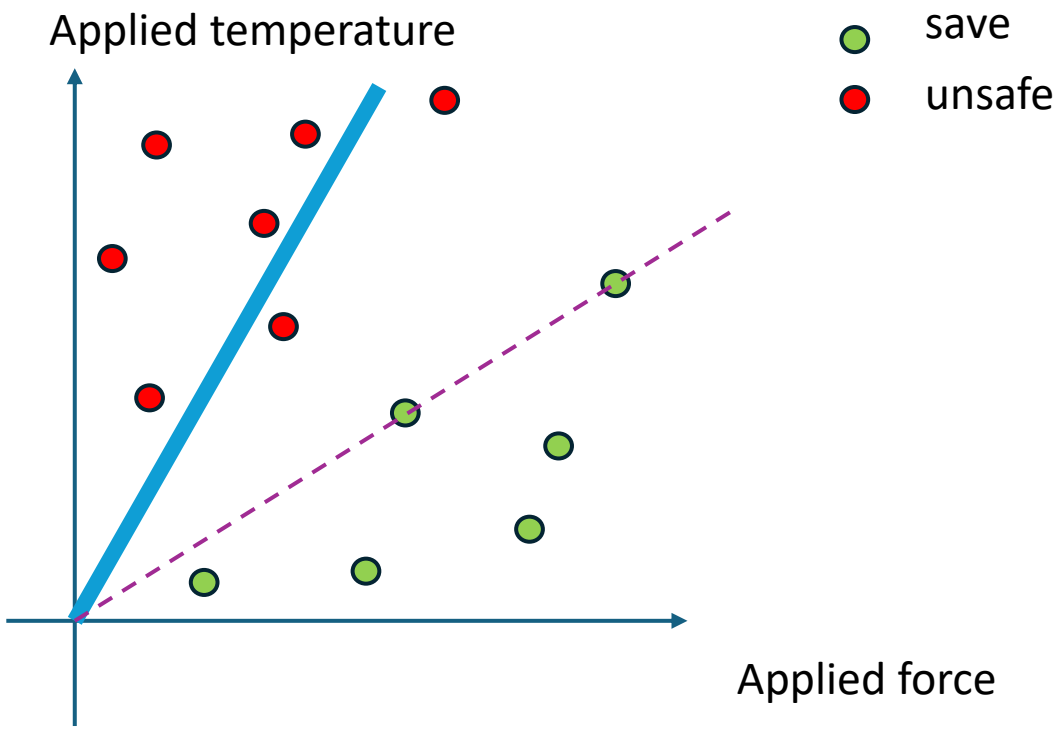
We now just need a suitable rate.



(for the current game there are maybe better algorithms)

Supervised Machine Learning

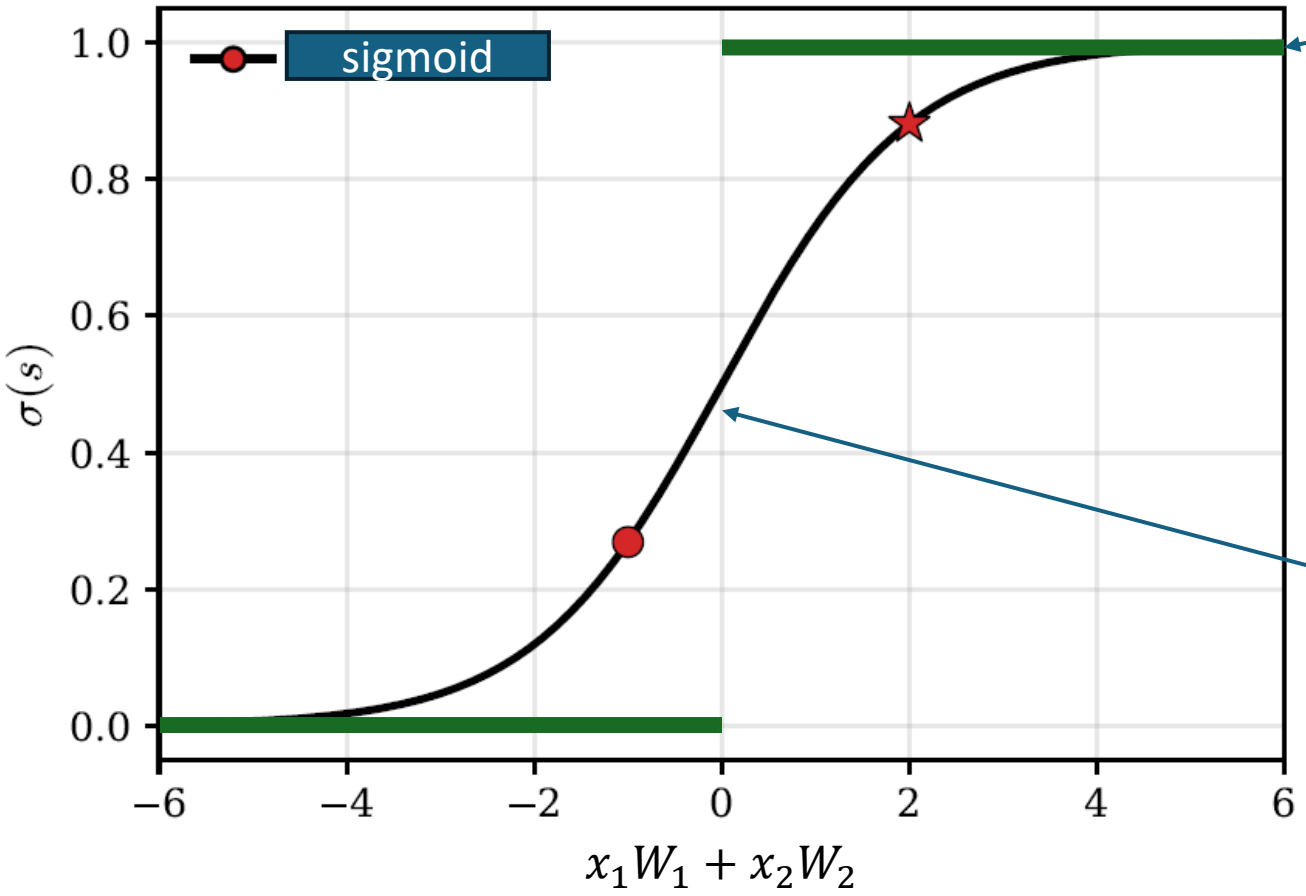
How is gradient descent helping us to find a save environment?
We search the weights that correspond to the minima of the loss (number of misclassifications)



Problem:
The derivate of the loss with respect to the weights is either zero or undefined, since either small weight changes do not change the loss or we see an jump of at least one up or down.

Changes to architecture and loss! (since the architecture and the loss are non differentiable functions of their input)

Q: Visualize old architecture



$$f_W(x) = \begin{cases} 1, x_1W_1 + x_2W_2 > 0 \\ 0 \quad , \quad else \end{cases}$$

$$= \begin{cases} 1, \ s(x_1, x_2) > 0 \\ 0 \quad , \ else \end{cases}$$

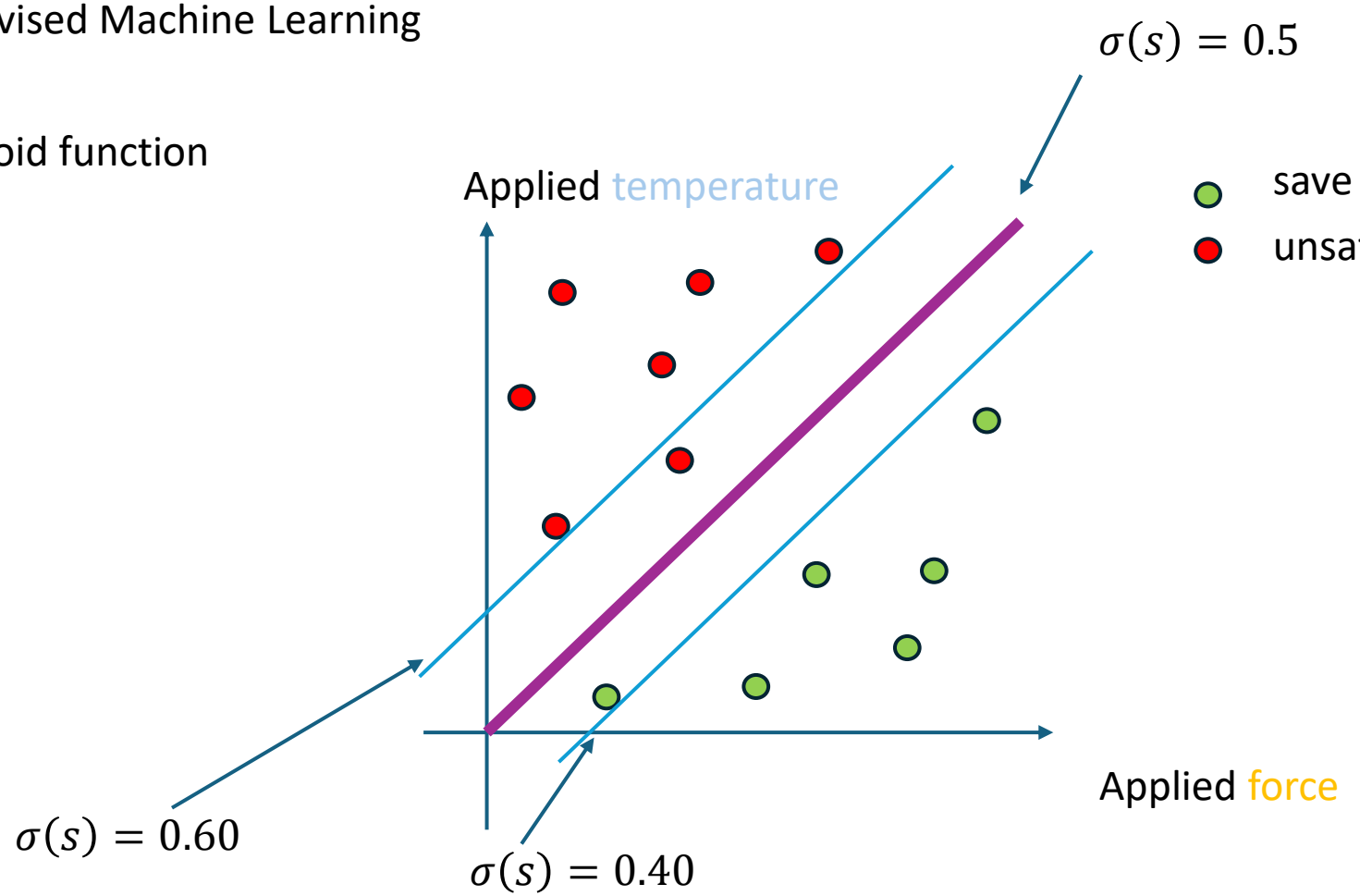
Sigmoid

$$\sigma(s) = \frac{1}{1 + e^{-x}}$$

Supervised Machine Learning

Illustration with sigmoid function

$s = w^T x$



Why are all lines parallel? What properties are useful to find an explanation for this?

A: σ is strictly monotone

B: s is linear in x

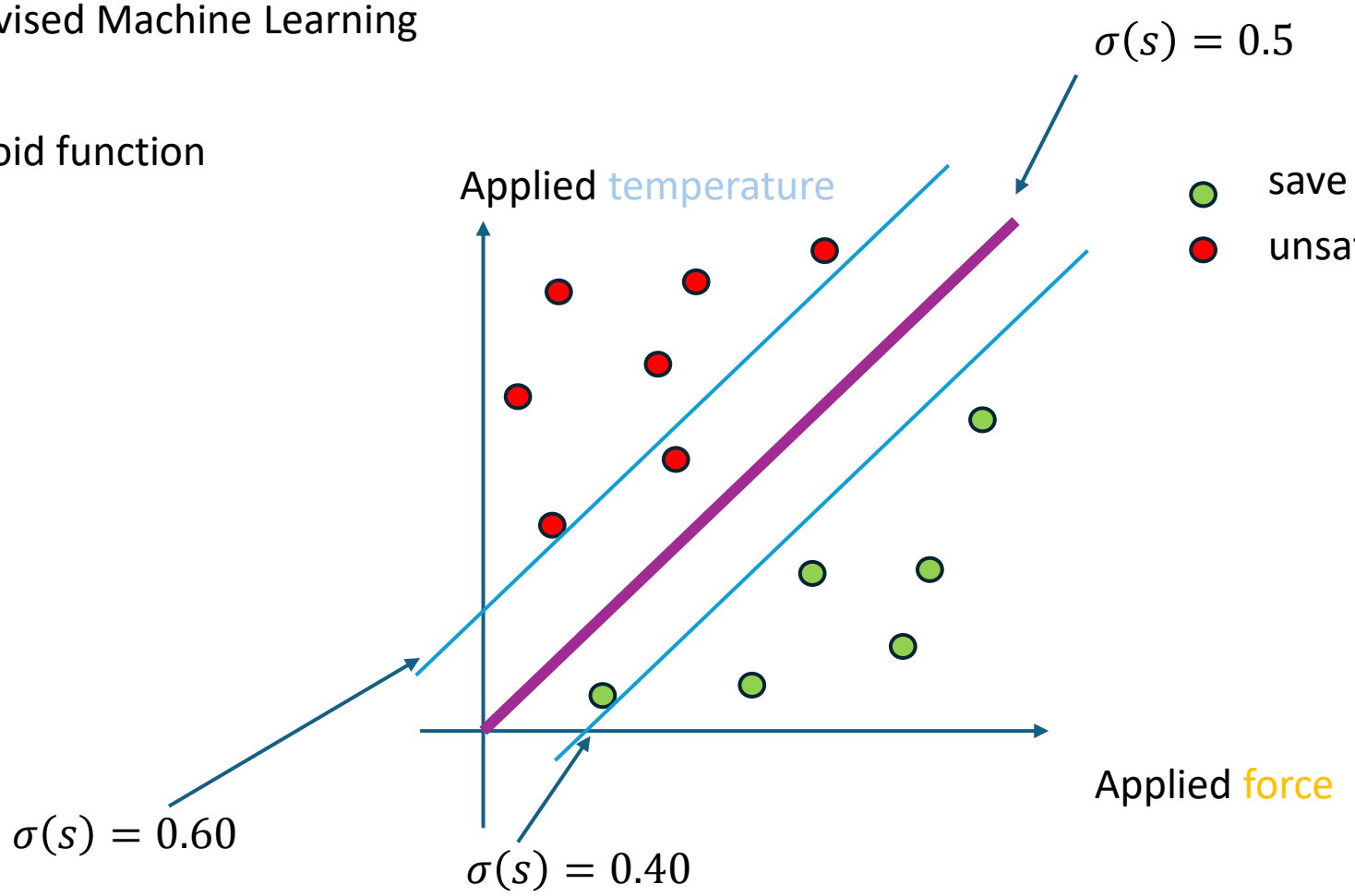
C: s is linear in w

D: the save and unsafe examples are linearly separable.

Supervised Machine Learning

Illustration with sigmoid function

$s = w^T x$

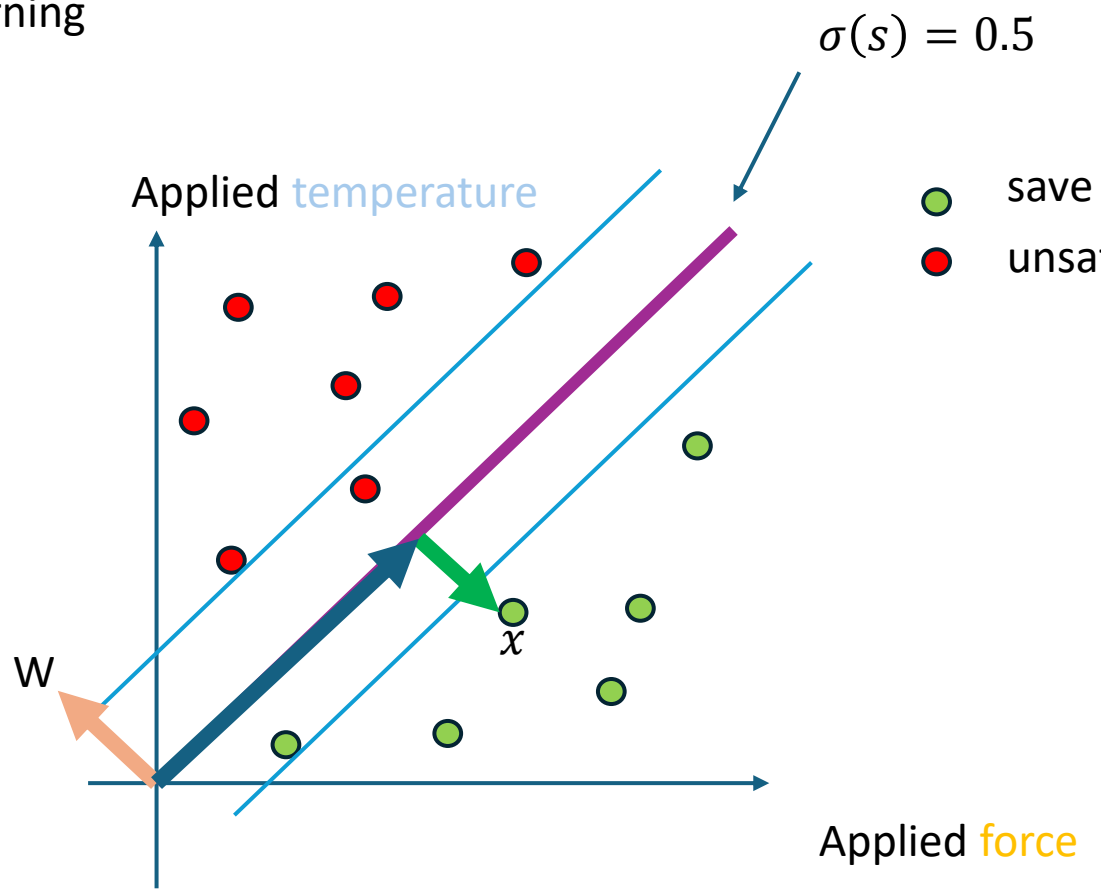


What property of W determines the angel of the lines?
What property of W determines the distance between the 0.6 and 0.4 line?

Supervised Machine Learning

Illustration with sigmoid function

$$s = w^T x$$

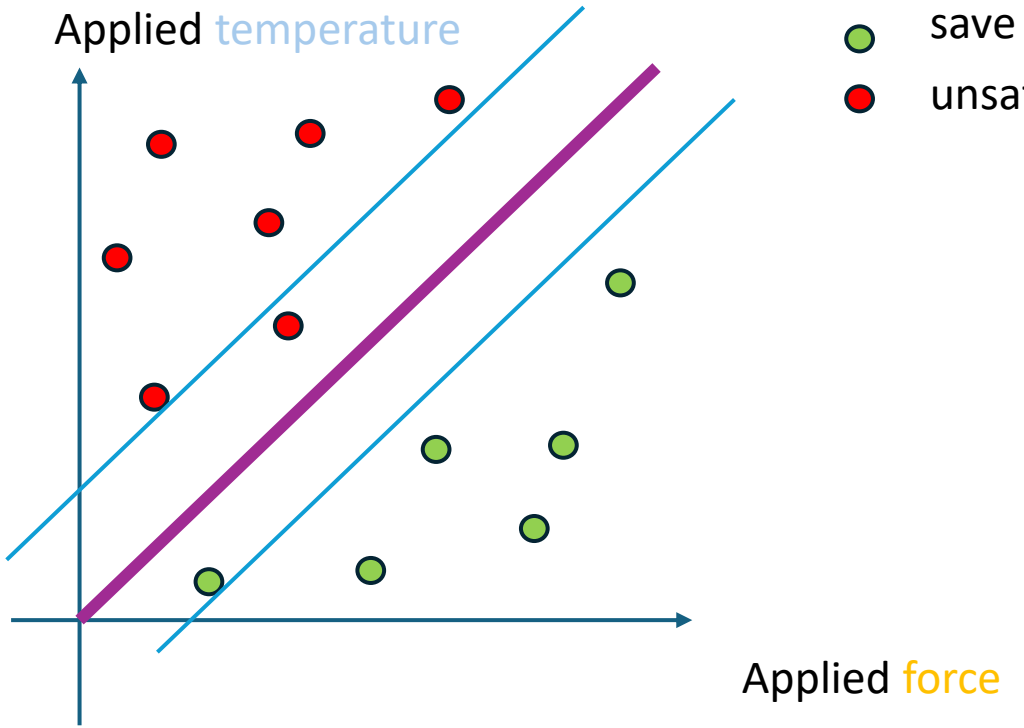


Asing the statements to the colours:
1) s stays constant when moving along this direction
2) Direction of maximal change.

Supervised Machine Learning

Motivation for loss:

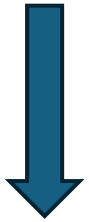
- If x_i is unsafe $\sigma\big(s(x)\big)$ should be close to 1
- If x_i is save $\sigma\big(s(x)\big)$ should be close to 0



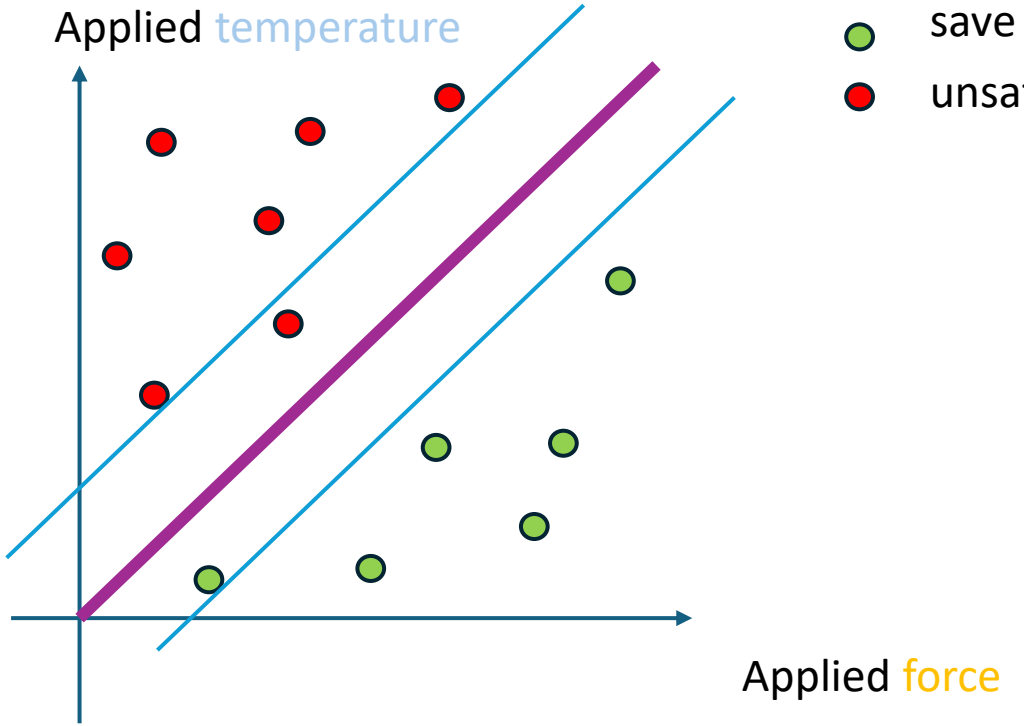
Supervised Machine Learning

Motivation for loss:

- If x_i is unsafe $\sigma\big(s(x)\big)$ should be close to 1 (that is its label y_i)
- If x_i is save $\sigma\big(s(x)\big)$ should be close to 0 (that is its label y_i)



A) $L(W) = \sum_{i=1}^{N}\big(\sigma\big(s_W(x_i)\big) - y_i\big)^2$



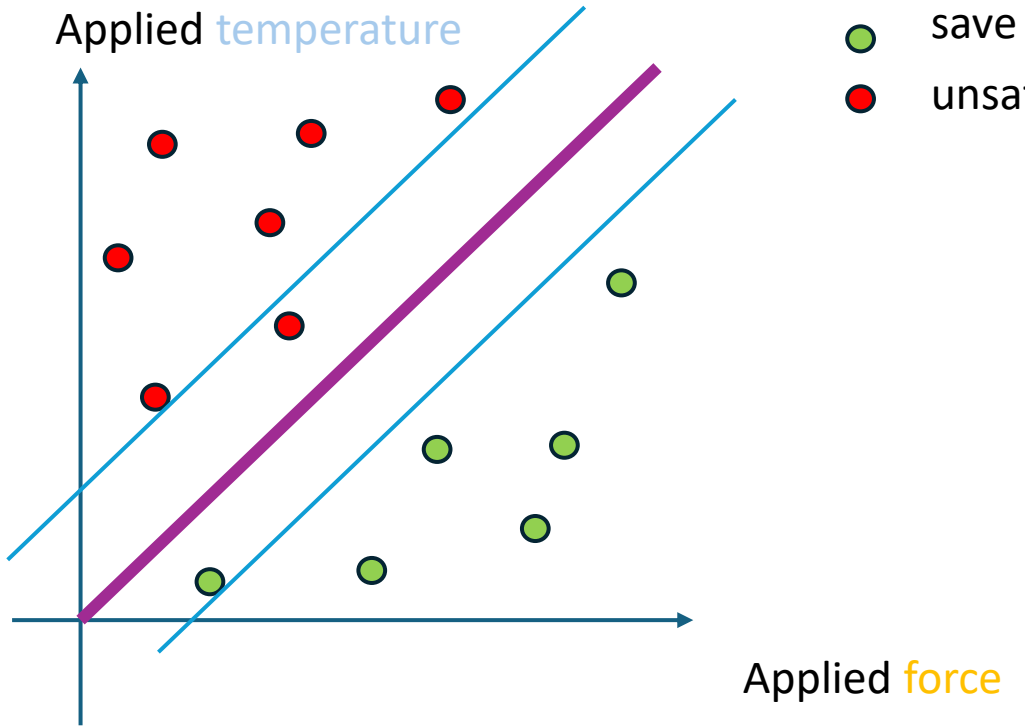
save

unsafe

Supervised Machine Learning

Motivation for loss:

- If x_i is unsafe $\sigma\big(s(x)\big)$ should be close to 1 (that is its label y_i)
- If x_i is save $\sigma\big(s(x)\big)$ should be close to 0 (that is its label y_i)



$$A)\ L(W) = \sum_{i=1}^{N}\big(\sigma\big(s_W(x_i)\big) - y_i\big)^2$$

$$B)\ L(W) = -\Big(\sum_{i=1}^{N} y_i \log\Big(\sigma\big(s_W(x_i)\big)\Big) + \sum_{i=1}^{N}(1-y_i)\log\Big(1-\sigma\big(s_W(x_i)\big)\Big)\Big)$$

Cross entropy loss,
generalizes to multi class,
has theoretic motivation,

Supervised Machine Learning

Assumption

$(x_i, y_i) \sim P_{X,Y}$

implies

Best label

$$P_{X,Y}(Y = y|X = x_i) = \frac{P_{X,Y}(Y = y, X = x_i)}{P_{X,Y}(X = x_i)}$$

NN returns:

$$Q(Y = y|X = x_i)$$

How good is the NN?

reformulation

How similar is Q to P?

Popular choice

the KL Divergence

$$D_{KL}(P||Q) = \sum_{c=1}^{C} p_i \log(p_i/q_i) = \sum_{c=1}^{C} p_i \log(p_i) - \sum_{c=1}^{C} p_i \log(q_i)$$

Entropy

Cross-Entropy

Q: What is the KL divergence of a distribution P to itself? $D_{KL}(P||P) =?$ $D_{KL}(P||P) = 0$, for all other distributions $D_{KL}(P||P) > 0$

Supervised Machine Learning

Assumption

$(x_i, y_i) \sim P_{X,Y}$

implies

Best label

$$P_{X,Y}(Y = y | X = x_i) = \frac{P_{X,Y}(Y = y, X = x_i)}{P_{X,Y}(X = x_i)}$$

NN returns:

$$Q(Y = y | X = x_i)$$

How good is the NN?

reformulation

How similar is Q to P?

Popular choice

the KL Divergence

$$D_{KL}(P||Q) = \sum_{c=1}^{C} p_i \log(p_i/q_i) = \sum_{c=1}^{C} p_i \log(p_i) - \sum_{c=1}^{C} p_i \log(q_i)$$

Entropy

Cross-Entropy

Q: What is $Q(Y = y | X = x_i)$ for the sigmoid?

We just define $Q(Y = 1 | X = x_i) = \sigma(s_W(x_i))$

What is $Q(Y = 0 | X = x_i) = ?$ $1 - \sigma(s_W(x_i))$

Supervised Machine Learning

Assumption

$(x_i, y_i) \sim P_{X,Y}$ implies

Best label

$$P_{X,Y}(Y = y|X = x_i) = \frac{P_{X,Y}(Y = y, X = x_i)}{P_{X,Y}(X = x_i)}$$

NN returns:

$$Q(Y = y|X = x_i)$$

How good is the NN? reformulation How similar is Q to P? Popular choice the KL Divergence

$$D_{KL}(P||Q) = \sum_{c=1}^{C} p_i \log(p_i/q_i) = \sum_{c=1}^{C} p_i \log(p_i) - \sum_{c=1}^{C} p_i \log(q_i)$$

Entropy

Cross-Entropy

Q: If we say

$$L(W) = \text{-1/N}(\sum_{i=1}^{N} y_i \log\left(\sigma\left(s_W(x_i)\right)\right) + \sum_{i=1}^{N}(1 - y_i) \log\left(1 - \sigma\left(s_W(x_i)\right)\right)) \approx E[CrossEnt(P|_x, Q_W|_X)]$$

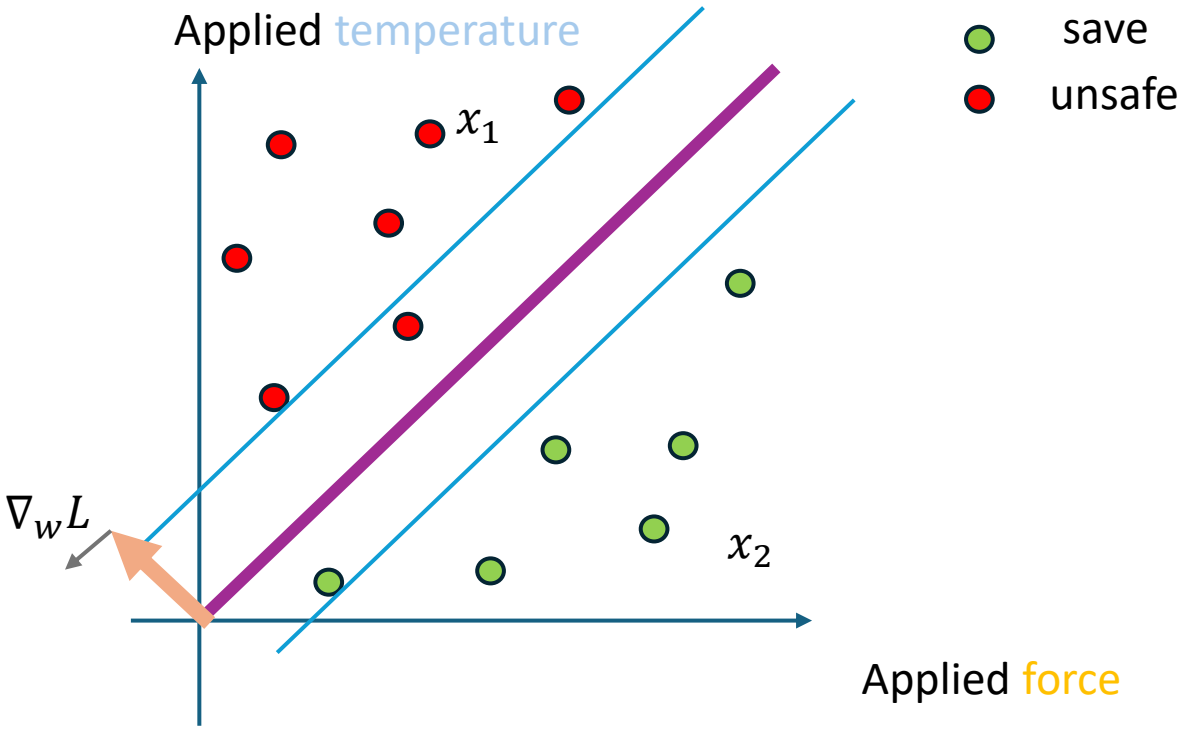
What is $P|_x$, ? $P(Y = 1|X = x_i) = \begin{cases} 1, y_i = ? \\ 0 \end{cases}$

Summary

We now have everything to approach the binary classification problem with ML

- Architecture: Sigmoid
- Loss: Cross entropy
- Way to update: gradient descent

But what is the gradient?



$$L(W) = \text{-1/N}\left(\sum_{i=1}^{N} y_i \log\left(\sigma\left(s_W(x_i)\right)\right) + \sum_{i=1}^{N}(1-y_i)\log\left(1-\sigma\left(s_W(x_i)\right)\right)\right) =$$

$$= \text{-1/N}\left[\log\left(\sigma\left(s_W(x_1)\right)\right) + \log\left(1-\sigma\left(s_W(x_2)\right)\right) + \ldots.\right]$$

Supervised Machine Learning

$$s_W(x_i) = w^T x_i$$

$$L(W) = \text{-1/N}\left(\sum_{i=1}^{N} y_i \log\left(\sigma\left(s_W(x_i)\right)\right) + \sum_{i=1}^{N}(1-y_i)\log\left(1-\sigma\left(s_W(x_i)\right)\right)\right) =$$

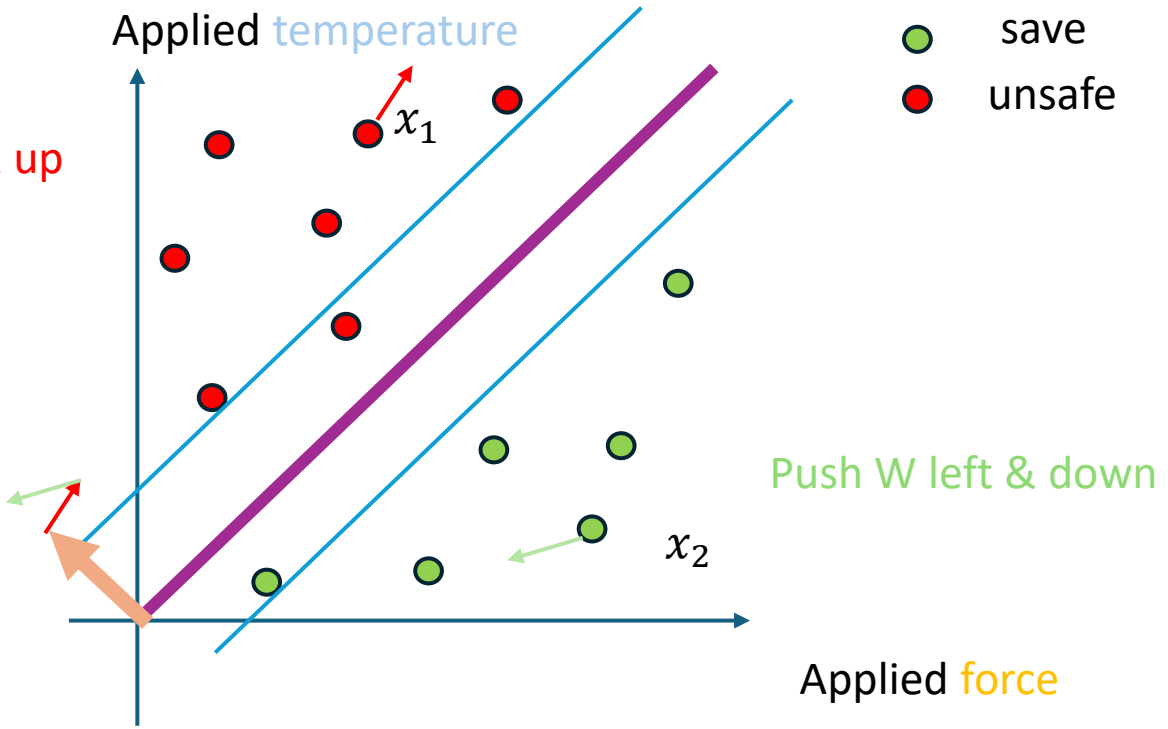
$$= \text{-1/N}\left[\log\left(\sigma\left(s_W(x_1)\right)\right) + \log\left(1-\sigma\left(s_W(x_2)\right)\right) + \ldots.\ \right]$$

For determining the grad note:
- $L(W)$ is a sum of N functions of W

- Each of this functions $l(x_i, y_i, W)$ is a concatenation of two functions

$$e(z, y_i) = 1/N \begin{cases} \log(z) & y_i = 1 \\ \log(1-z) & y_i = 0 \end{cases}$$

$$z(x_i, W) = \sigma\left(s_W(x_i)\right)$$



Chain rule:

$$\nabla_W l(x_i, y_i, W) = \nabla_z\, e(n(x_i, W), y_i)\, \nabla_W[\sigma\left(s_W(x_i)\right)] = 1/N \begin{cases} 1/z & y_i = 1 \\ -1/(1-z) & y_i = 0 \end{cases} \sigma(s)(1-\sigma(s))\nabla_W\left[\left(s_W(x_i)\right)\right]$$

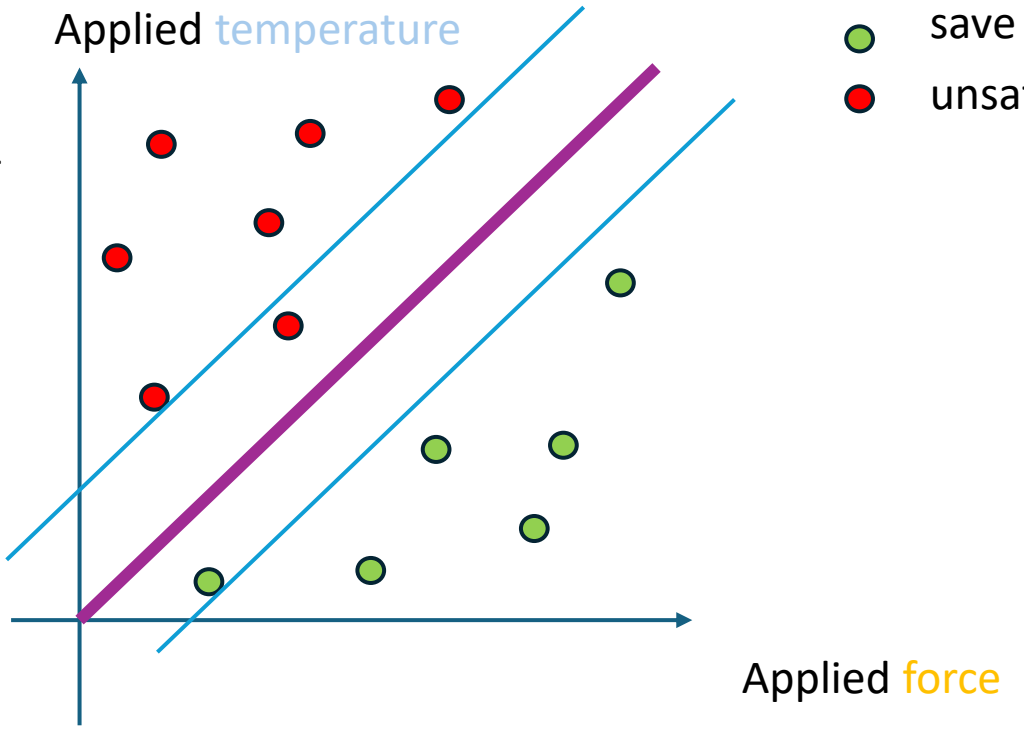
$$= x_i/N \begin{cases} (1-\sigma(s(x_i))) & y_i = 1 \\ -\sigma(s(x_i)) & y_i = 0 \end{cases}$$

Sing depends on true label, magnitude on predicted label, errors get highest magnitude.

Supervised Machine Learning

We now have everything to approach the binary classification problem with ML

- Architecture: Sigmoid
- Loss: Cross entropy
- Way to update: gradient descent

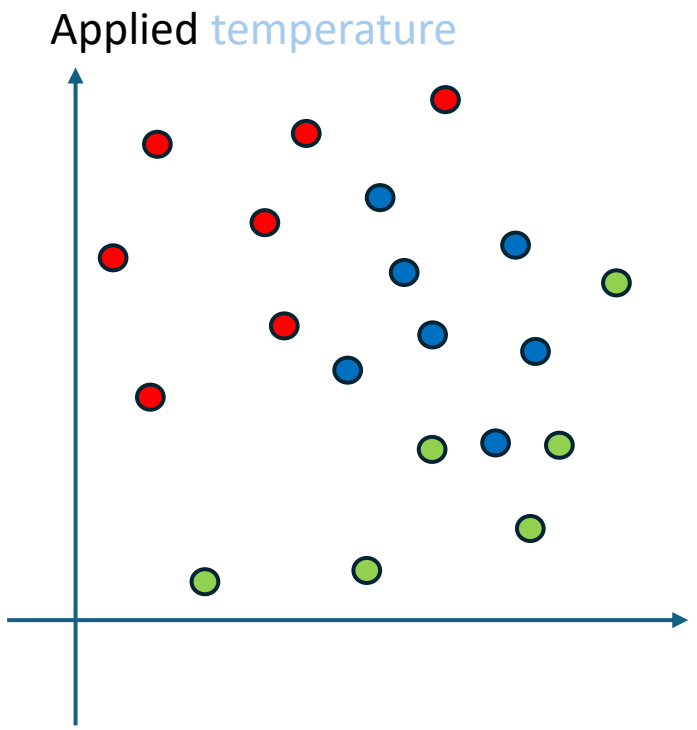


Next step: More then two classes

Supervised Machine Learning

Next step: More then two classes

With sigmoid we can only do binary!

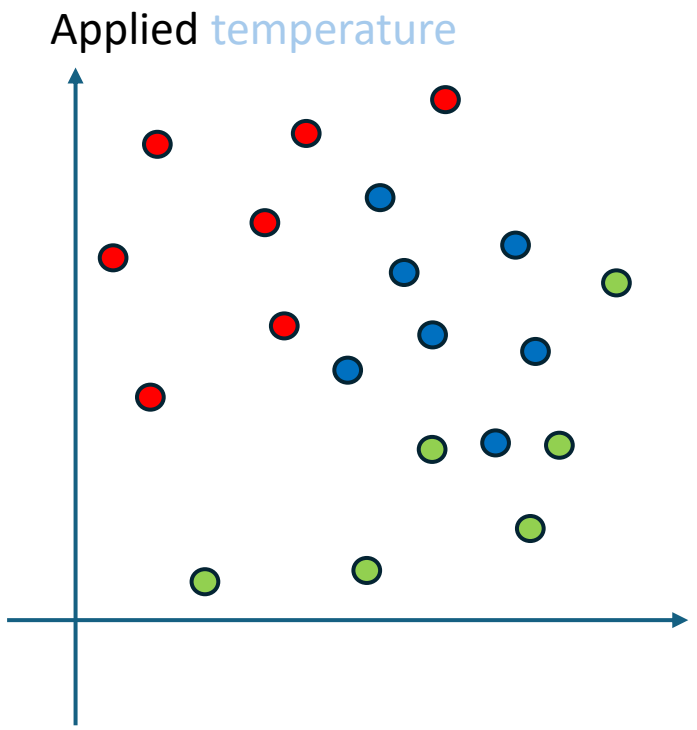


- save
- unsafe
- needs extra care

Supervised Machine Learning

Next step: More then two classes

SoftMax can do multi class:
Therefor we:
- change s, we map x to a 3D vector
- apply sigmoid to each dimension
- map this 3D vector to a probability vector

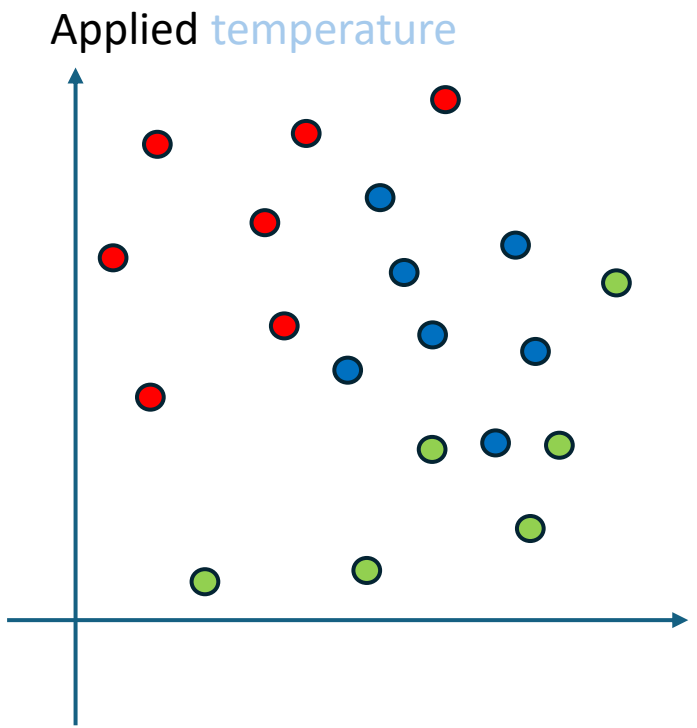


save
unsafe
needs extra care

Supervised Machine Learning

Next step: More then two classes

SoftMax can do multi class:
Therefor we:
- **change s, we map x to a 3D vector**
- apply sigmoid to each dimension
- map this 3D vector to a probability vector



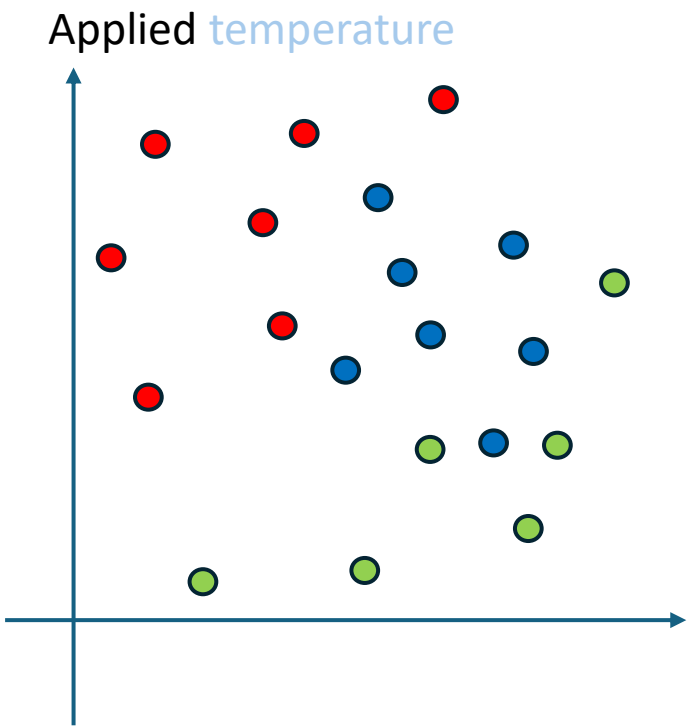
save
unsafe
needs extra care

$$s_W(x)=\begin{cases} W_{1,1}x_1 + W_{1,2}x_2 \\ W_{2,1}x_1 + W_{2,2}x_2 = Wx \\ W_{3,1}x_1 + W_{3,2}x_2 \end{cases}$$

Supervised Machine Learning

Next step: More then two classes

SoftMax can do multi class:
Therefor we:
- change s, we map x to a 3D vector
- apply sigmoid to each dimension
- map this 3D vector to a probability vector



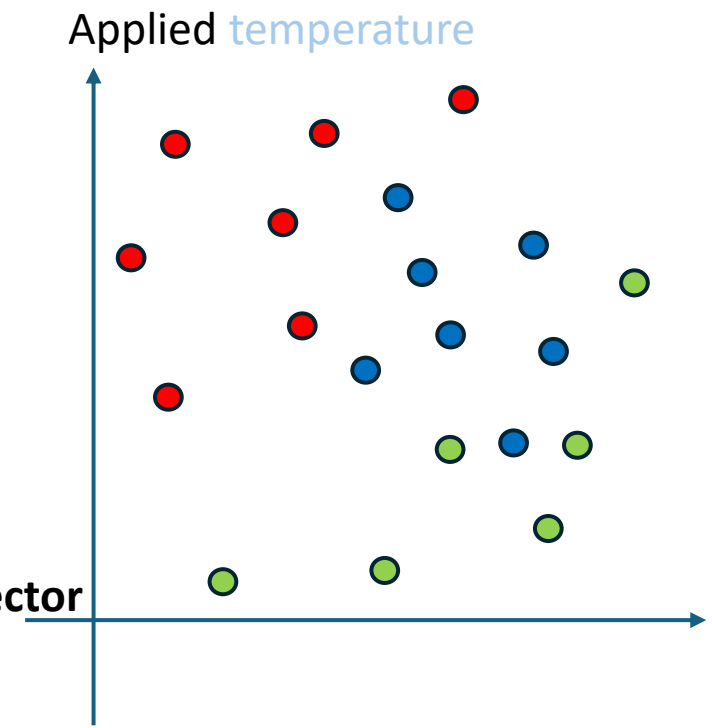
save
unsafe
needs extra care

$$s_W(x)=\begin{cases} W_{1,1}x_1 + W_{1,2}x_2 \\ W_{2,1}x_1 + W_{2,2}x_2 = Wx \\ W_{3,1}x_1 + W_{3,2}x_2 \end{cases}$$

Let $z = \sigma(s)$

Supervised Machine Learning

Next step: More then two classes



save
unsafe
needs extra care

SoftMax can do multi class:
Therefor we:
Therefor we:
- change s, we map x to a 3D vector
- apply sigmoid to each dimension
- map this 3D vector to a probability vector

$$s_W(x)=\begin{cases} W_{1,1}x_1 + W_{1,2}x_2 \\ W_{2,1}x_1 + W_{2,2}x_2 = Wx \\ W_{3,1}x_1 + W_{3,2}x_2 \end{cases}$$

Summs to one

$$\text{SoftMax}(z)_j = \frac{e^{z_j}}{\sum_{k=1}^{3} e^{z_k}}$$ For each class j=1,2,3

Where $z = \sigma(s)$

Supervised Machine Learning

$$s_W(x)=\begin{cases} W_{1,1}x_1 + W_{1,2}x_2 \\ W_{2,1}x_1 + W_{2,2}x_2 = Wx \\ W_{3,1}x_1 + W_{3,2}x_2 \end{cases}$$

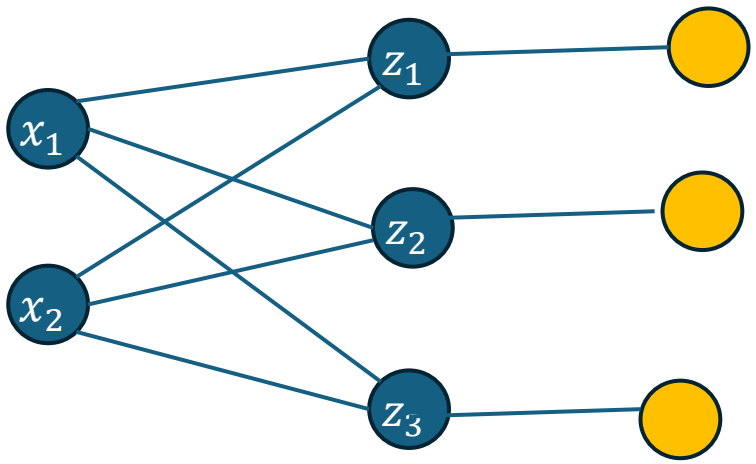
Summs to one

$$\sigma(s)_j = \frac{e^{s_j}}{\sum_{k=1}^{3} e^{s_k}}$$

For each class j=1,2,3

Where $z = \sigma(s)$

ML uses a **graphical language** for representing this formulas



Supervised Machine Learning

$$s_W(x)=\begin{cases} W_{1,1}x_1 + W_{1,2}x_2 \\ W_{2,1}x_1 + W_{2,2}x_2 = Wx \\ W_{3,1}x_1 + W_{3,2}x_2 \end{cases}$$

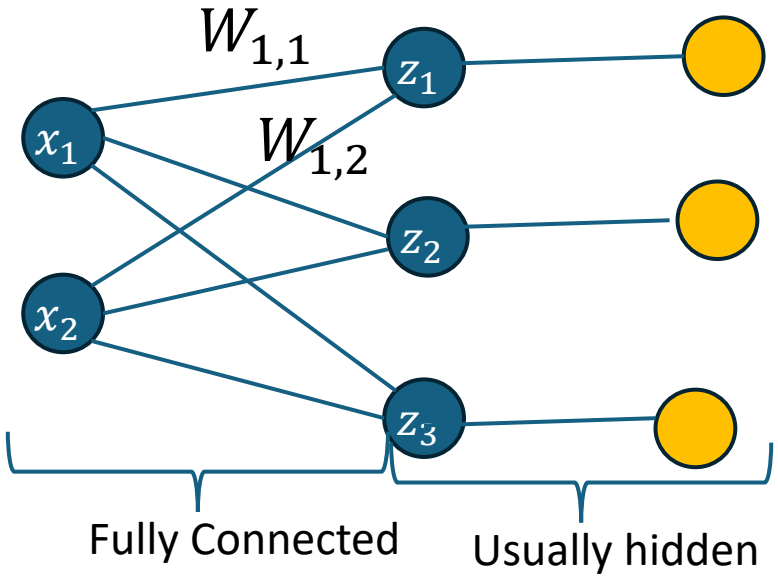
Summs to one

$$\sigma(s)_j = \frac{e^{s_j}}{\sum_{k=1}^{3} e^{s_k}}$$

For each class j=1,2,3

Where $z = \sigma(s)$

ML uses a graphical language for representing this formulas



Supervised Machine Learning

$$s_W(x)=\begin{cases} W_{1,1}x_1 + W_{1,2}x_2 \\ W_{2,1}x_1 + W_{2,2}x_2 = Wx \\ W_{3,1}x_1 + W_{3,2}x_2 \end{cases}$$

Is called logits

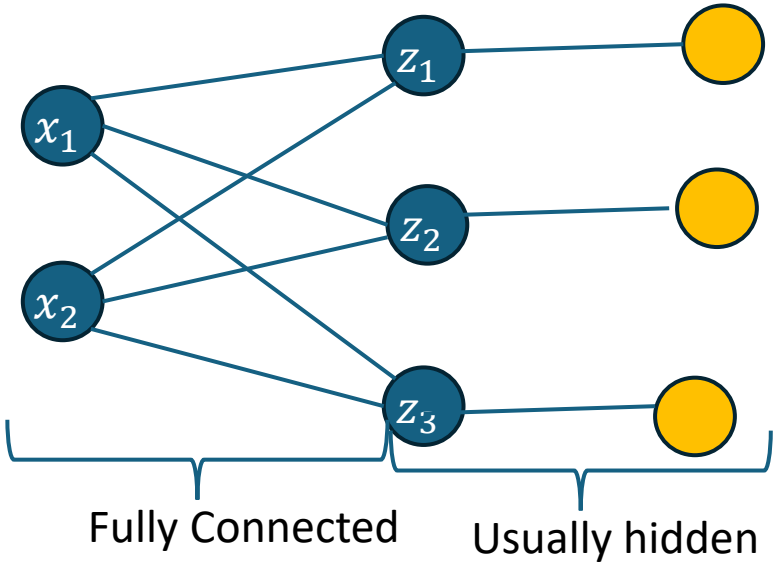
Summs to one

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^{3} e^{z_k}}$$

For each class j=1,2,3

Where $z = \sigma(s)$

ML uses a graphical language for representing this formulas

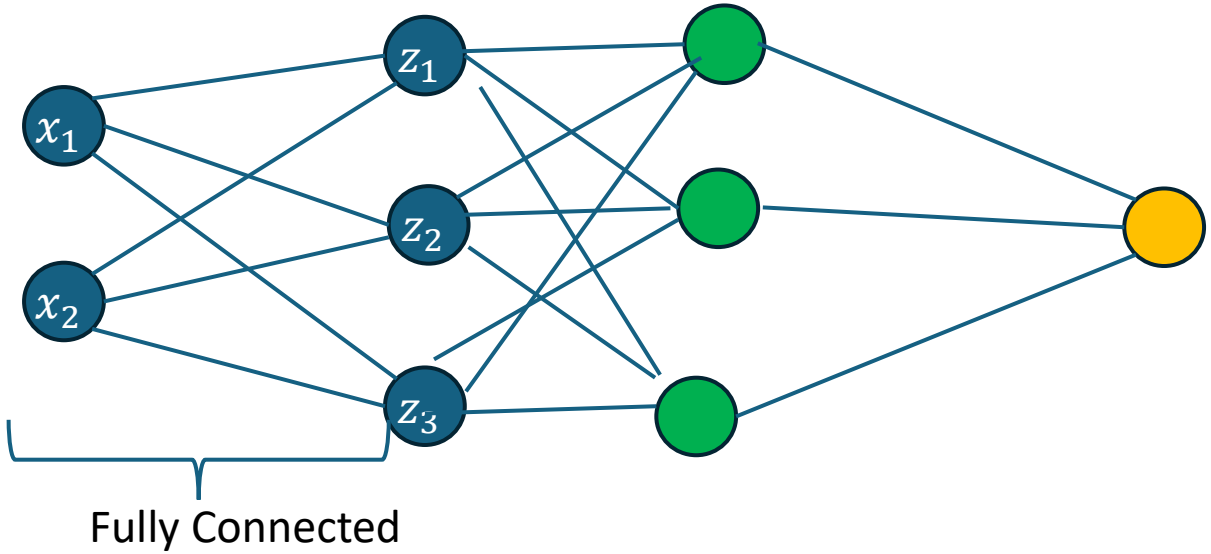


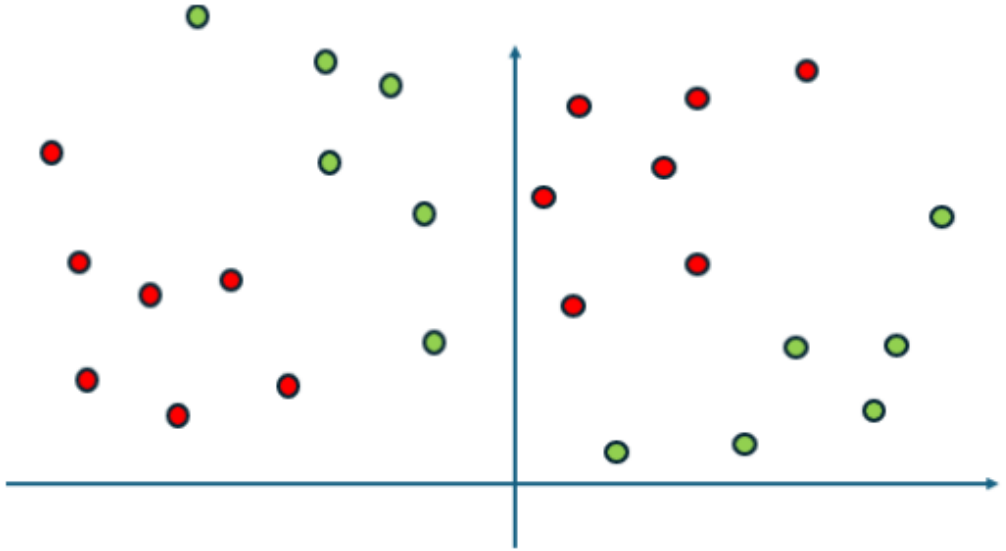
Loss:

$$L(W) = L(W; \{(x_i, y_i)\}_{i=1}^{N}) = -\sum_{i=1}^{N}\sum_{c=1}^{3} 1_{y_i=c} \log(\sigma(s(x_i))_c)$$

Supervised Machine Learning

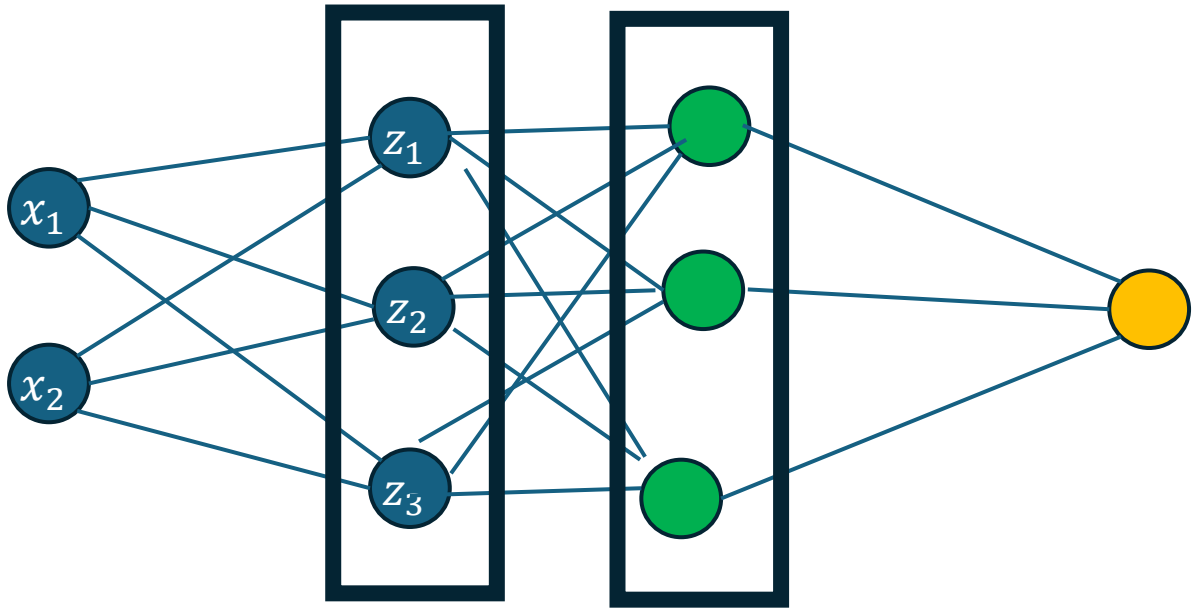
Solution for not linearly separatable problems: More layers!



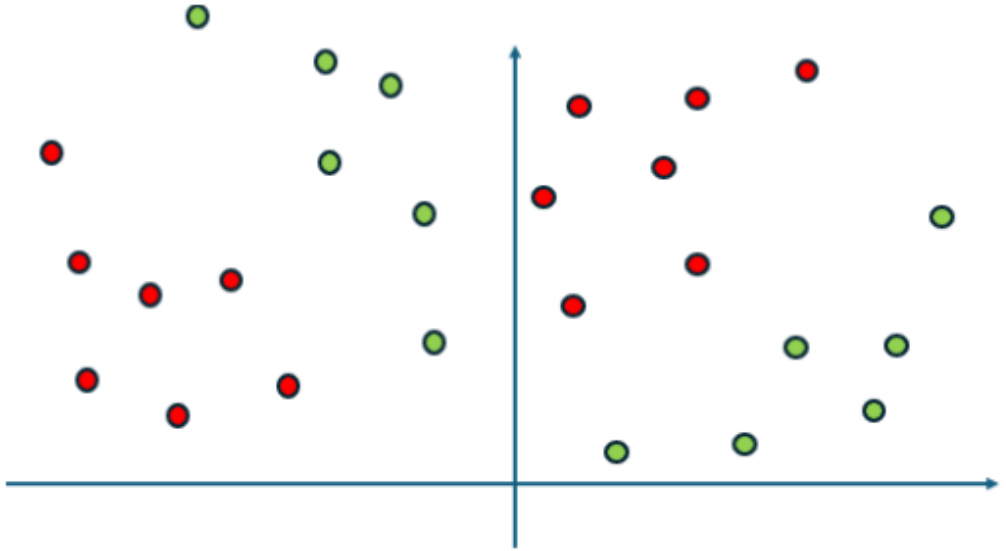


Supervised Machine Learning

Solution for not linearly separatable problems: More layers!



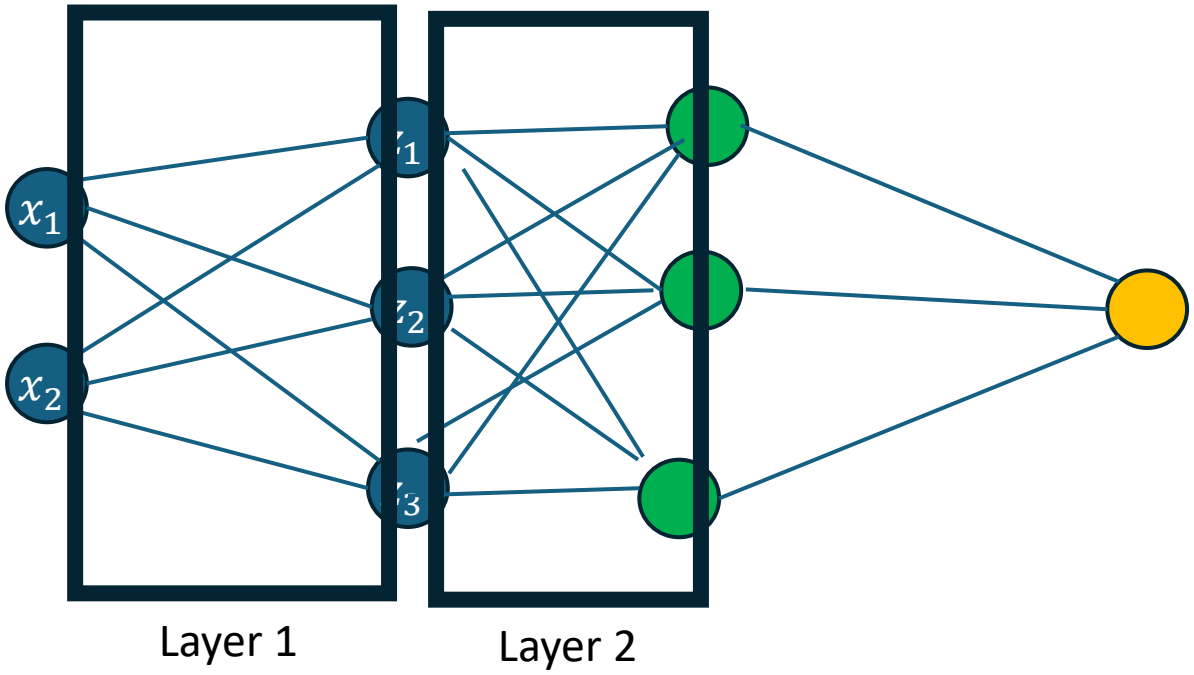
Representation 1 Representation 2

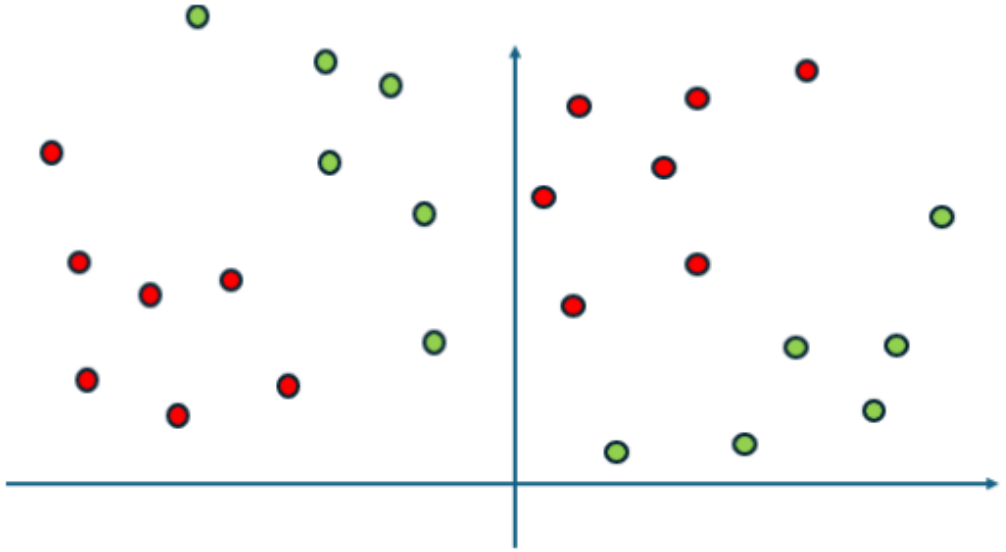


The output is the highest level representation.

Supervised Machine Learning

Solution for not linearly separatable problems: More layers!





$$f(x, W) = \mathrm{f}_3(\mathrm{f}_2(\mathrm{f}_1(\mathrm{x}, \mathrm{W}_1), \mathrm{W}_2), \mathrm{W}_3)$$

The mapping between two representations is call layer. (Here literature is not always consistent)

Supervised Machine Learning

Learning = optimize W so that it fits to the data

$$L(W) = L(W; \{(x_i, y_i)\}_{i=1}^{N}) = \sum_{i=1}^{N} \sum_{c=1}^{3} 1_{y_i=c} \log(NN(x_i)_c)$$

Dimension growth with the number of layers and the hidden features per layer

Gradient Descent

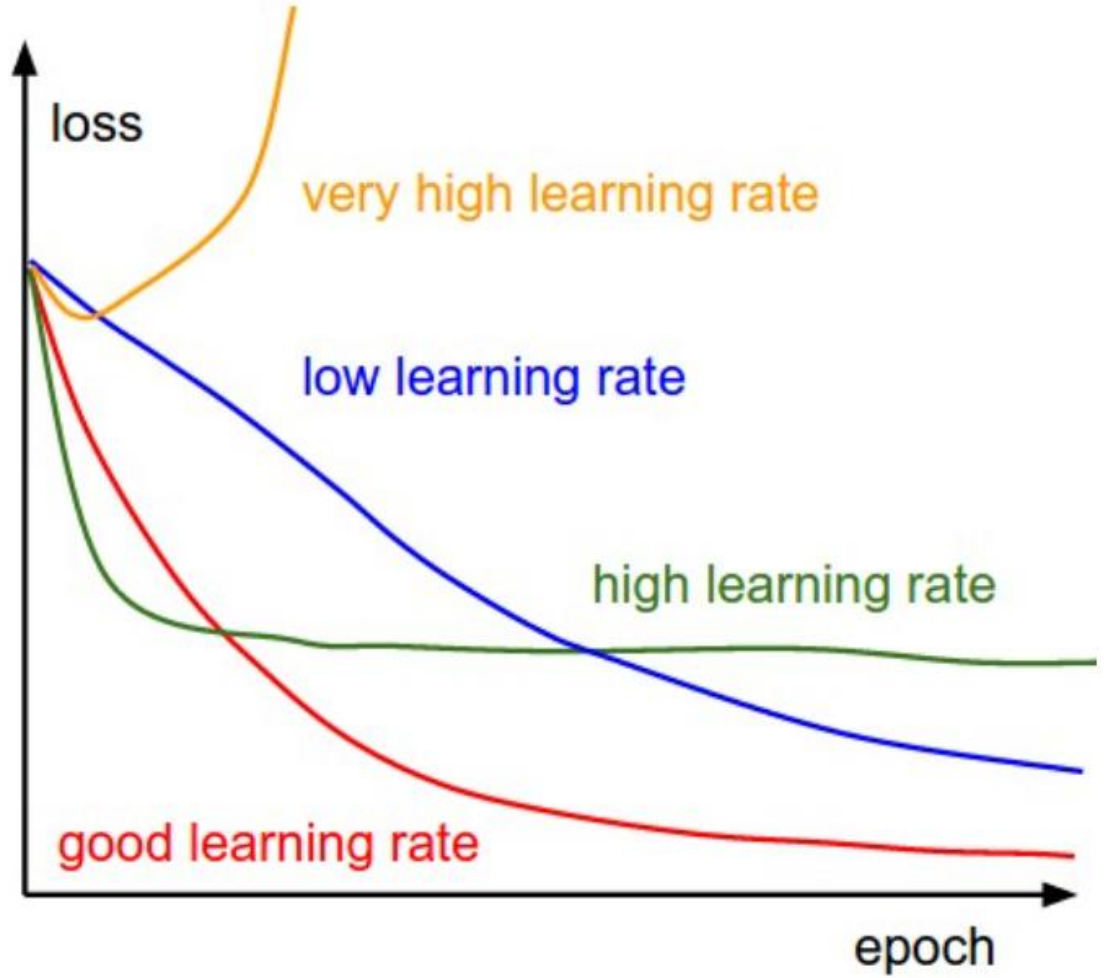
We worry later, PyTorch will do it for us anyway

1. Start at some W, for example W=0.

2. Determine the gradient g(W) and update x to W-μg.

3. If μ is small enough then f(W-μg)<f(W).

How to choose this learning rate



Assing to the colours:

A) Good learning rate
B) High learning rate
C) Low learning rate
D) Very hight learning rate

1 = ? 2=? 3=? 4=?

Epoch

Left: A cartoon depicting the effects of different learning rates. With low learning rates the improvements will be linear. With high learning rates they will start to look more exponential. Higher learning rates will decay the loss faster, but they get stuck at worse values of loss (green line). This is because there is too much "energy" in the optimization and the parameters are bouncing around chaotically, unable to settle in a nice spot in the optimization landscape.

CS231n Deep Learning for Computer Vision

Supervised Machine Learning

Now we worry about the gradient.

$$L(W) = L(W; \{(x_i, y_i)\}_{i=1}^N) = -\sum_{i=1}^N \sum_{c=1}^3 \ 1_{y_i=c} \ \log(NN(x_i)_c)$$

We use the linearity of the gradient [the gradient of the sum of functions is the sum of the gradients of this functions]

$$\nabla_W L(W) = -\sum_{i=1}^N \sum_{c=1}^3 \ 1_{y_i=c} \ \nabla_W \log(NN(x_i)_c) = -\sum_{i=1}^N \sum_{c=1}^3 \ 1_{y_i=c} \ \nabla_W (NN(x_i)_c)/NN(x_i)_c$$

Q: Gradient descent will change the W so that

A: W has more zero entries

B: On average the prediction for x_i is more concentred on y_i

C: predictions that are concentrated on the wrong label see the biggest change

Multiple are true

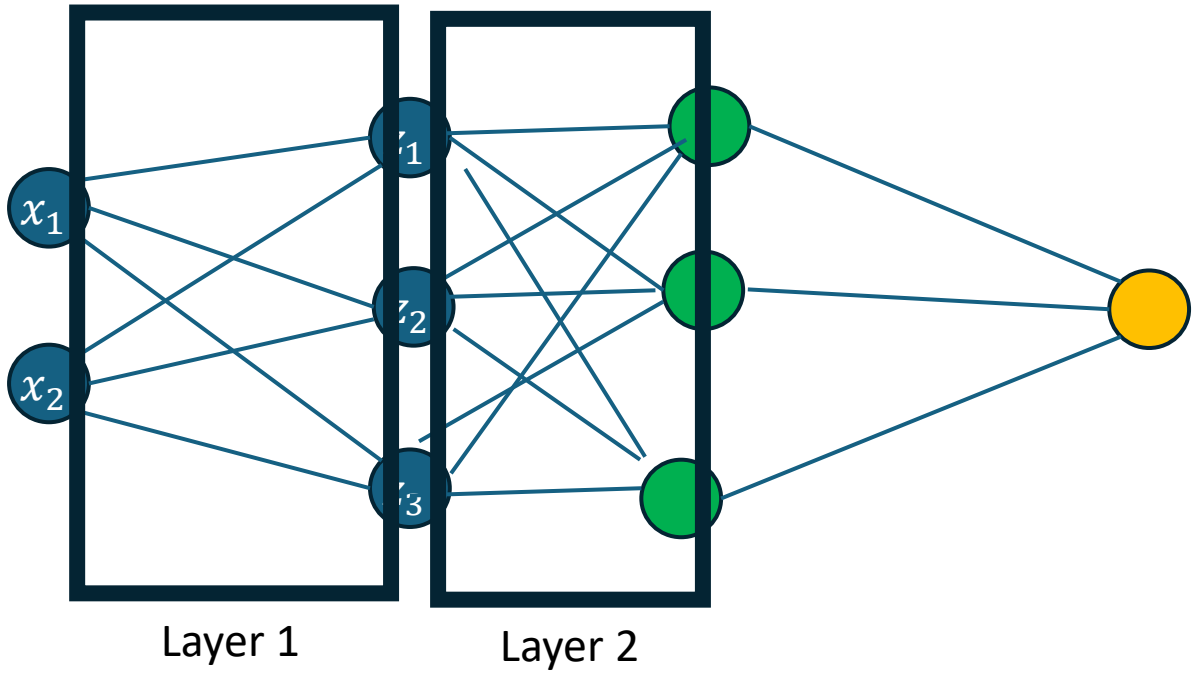
next

$$\nabla_W (NN(x_i)_c = ?$$

Supervised Machine Learning

How to determine the gradient?

Gradient is the collection of all partial derivatives

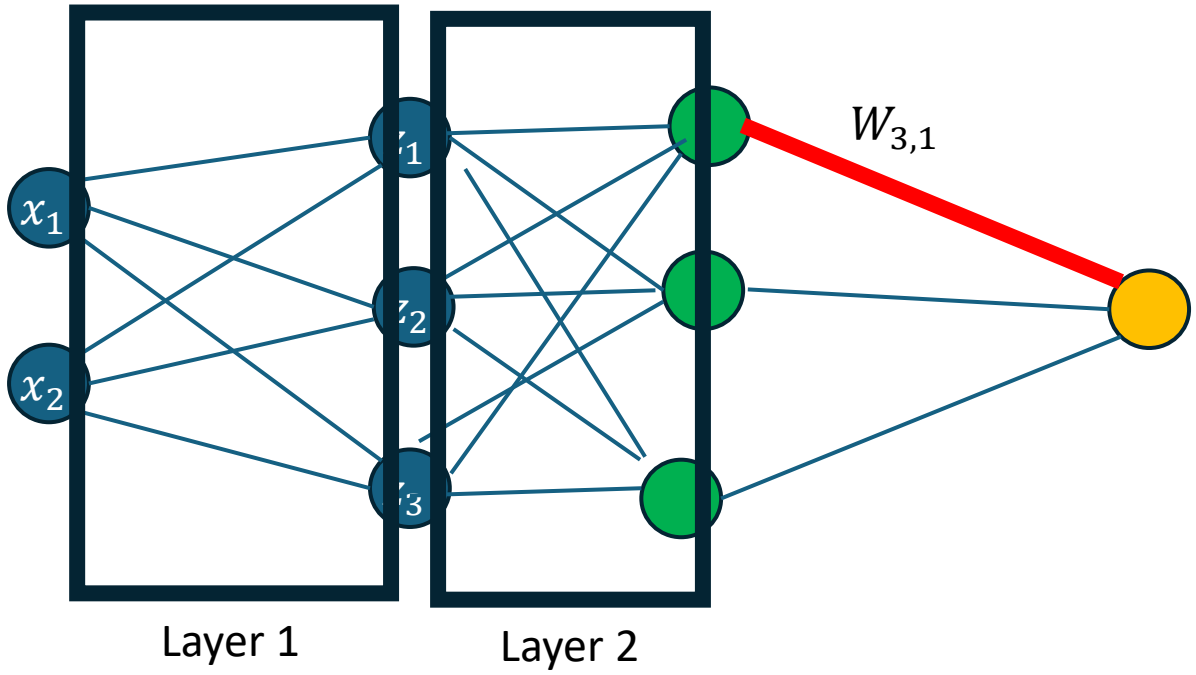


$$f(x, W) = \mathrm{f}_3(\mathrm{f}_2(\mathrm{f}_1(\mathrm{x}, \mathrm{W}_1), \mathrm{W}_2), \mathrm{W}_3)$$

Supervised Machine Learning

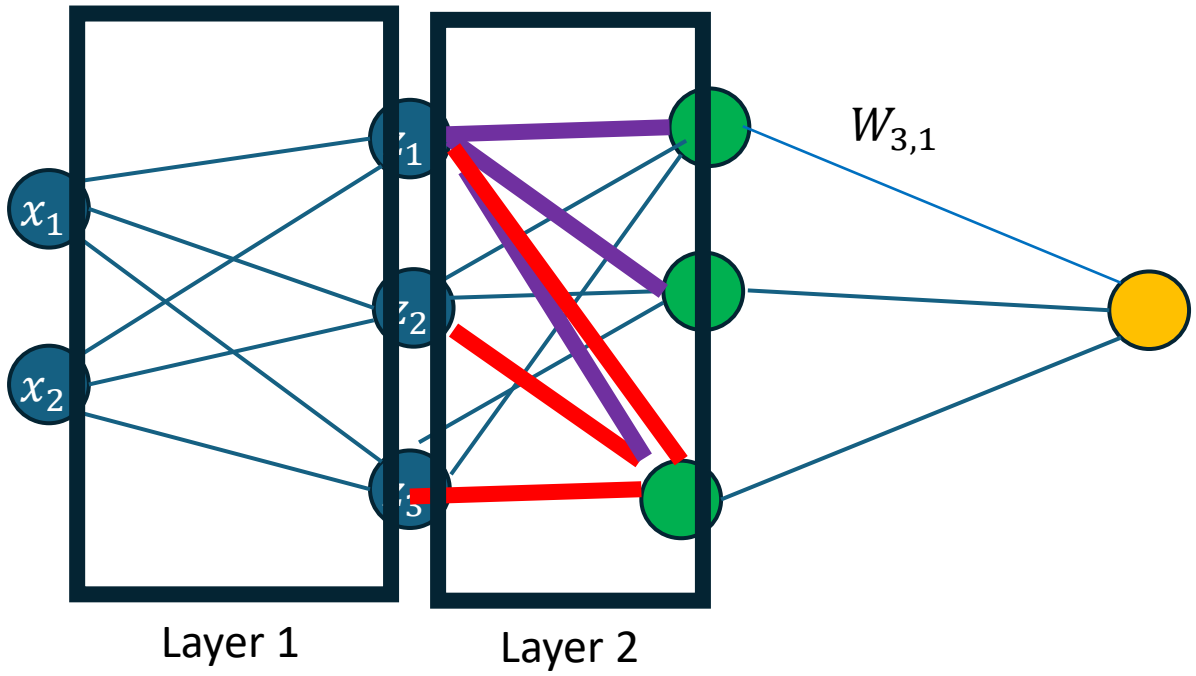
How to determine the gradient?

$$\frac{\partial}{\partial W_{3,1}} f(x, W) = \frac{\partial}{\partial W_{3,1}} \mathrm{f}_3(\mathrm{h(x)}, \mathrm{W}_3)$$



Supervised Machine Learning

How to determine the gradient?



$$\frac{\partial}{\partial W_{3,1}} f(x, W) = \frac{\partial}{\partial W_{3,1}} \mathrm{f_3}(\mathrm{h(x, W)}, \mathrm{W_3})$$

$$\frac{\partial}{\partial W_{3,2}} f(x, W) = \frac{\partial}{\partial W_{3,2}} \mathrm{f_3}(\mathrm{h(x, W)}, \mathrm{W_3})$$

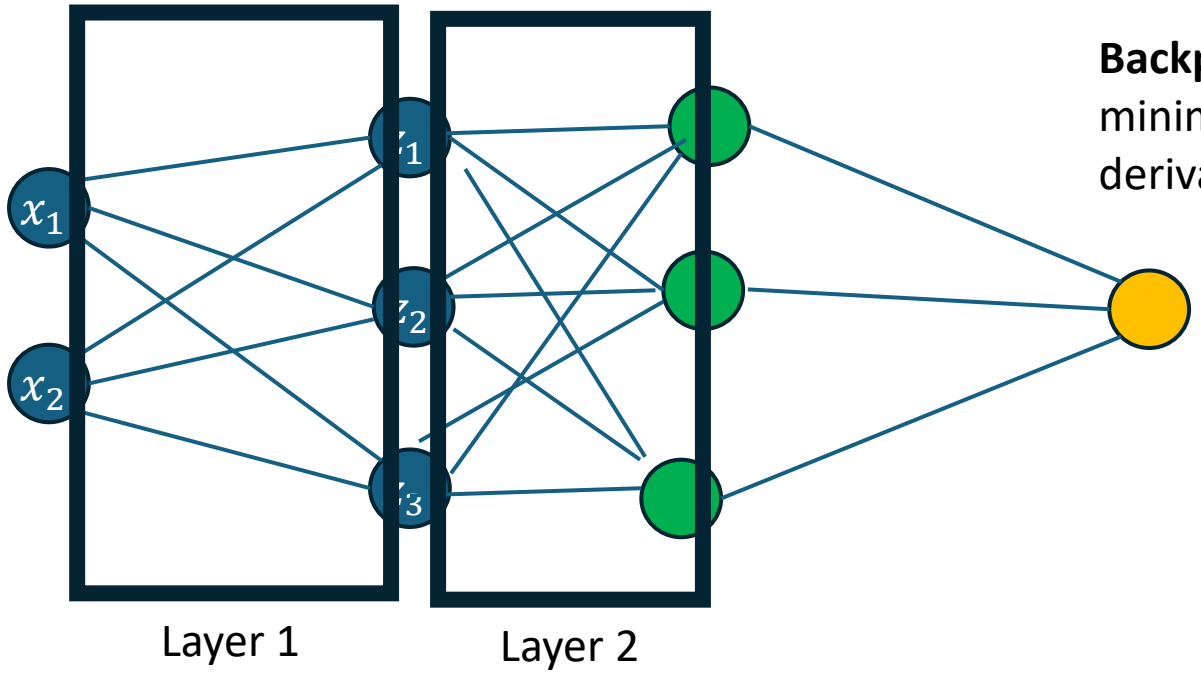
....

Where do we duplicate work?

A: B:

Supervised Machine Learning

How to determine the gradient?

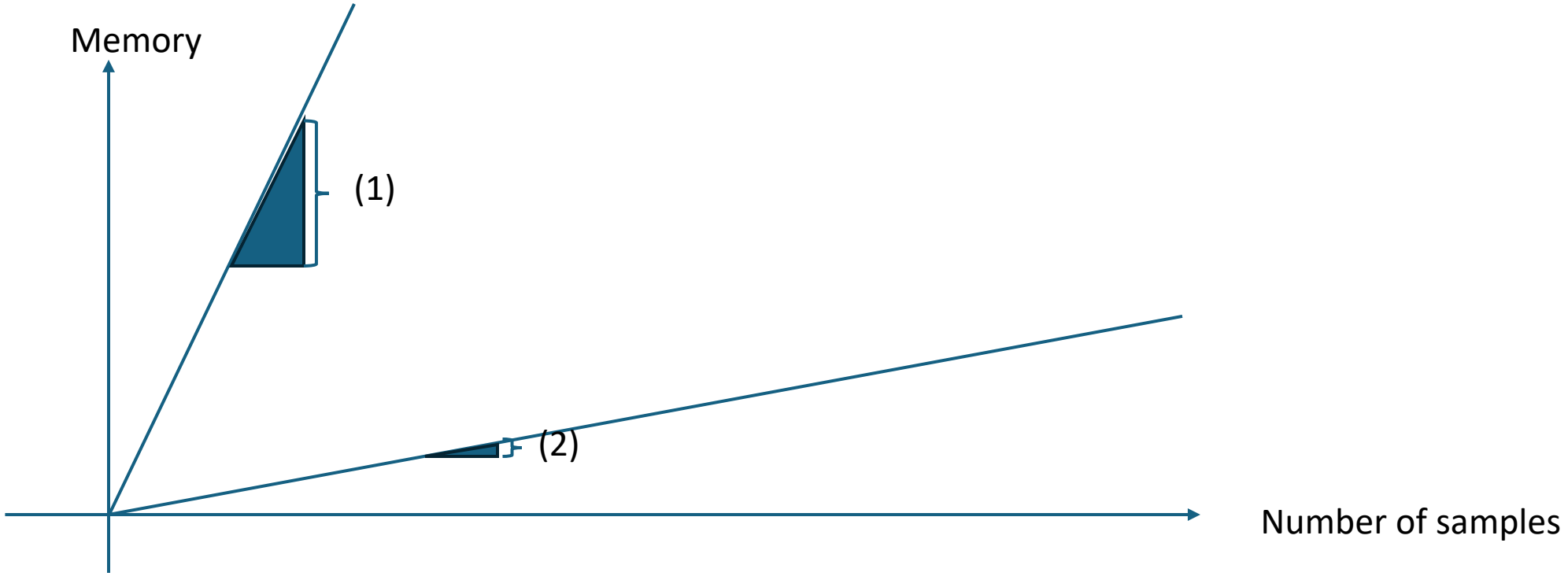


Gradient is the collection of all partial derivatives

$$f(x, W) = \mathrm{f}_3(\mathrm{f}_2(\mathrm{f}_1(\mathrm{x}, \mathrm{W}_1), \mathrm{W}_2), \mathrm{W}_3)$$

Backpropagation: A way to calculate the partial derivatives, that minimizes the number of repeated calculations by saving the derivates with respect to the internal layers.

Stochastic gradient descent



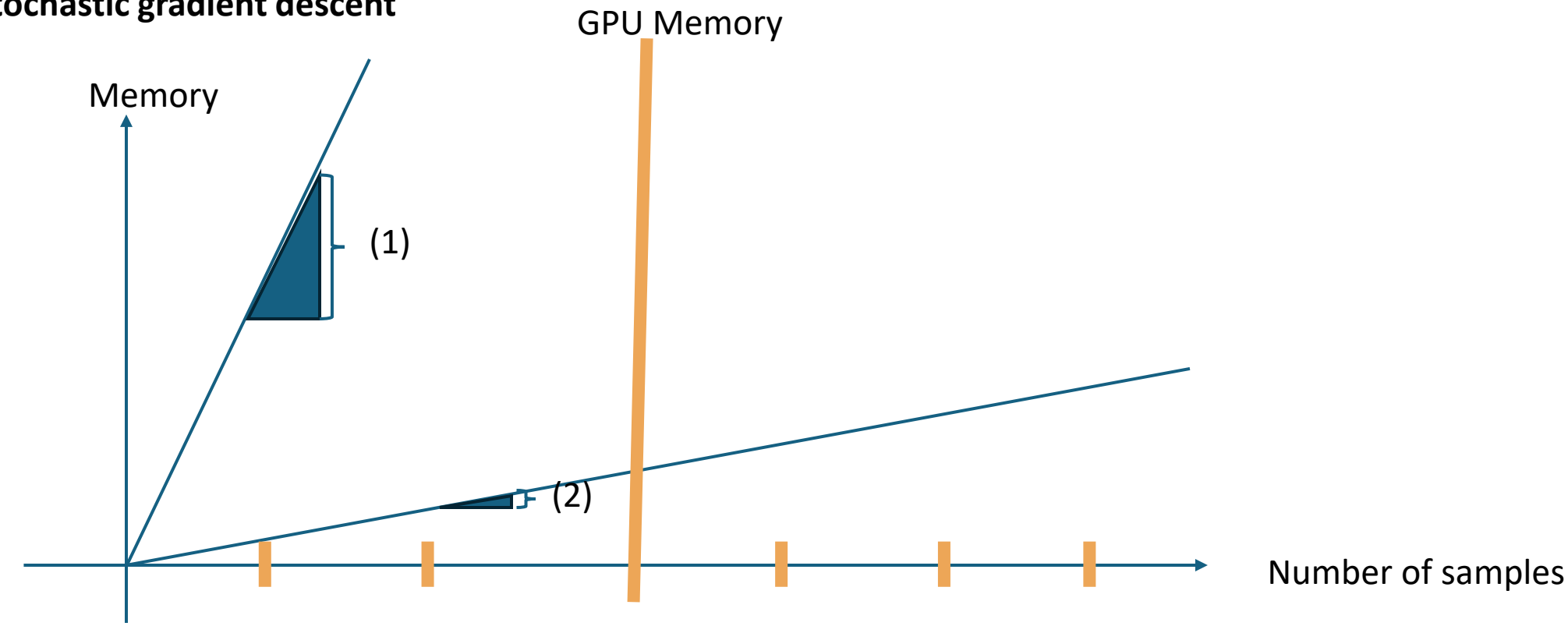
Q: (1) & (2) are determined by

A) Number of neurons

B) Slope of the sigmoid

C) Size of the GPU

Stochastic gradient descent



We split the data into **multiple batches** and calculate the gradient on the first batch.

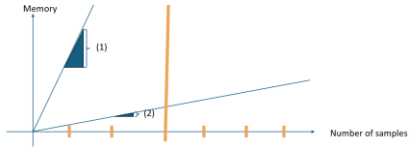
Then we do a step in the direction of this gradient.

Then we calculate the gradient on the second batch.

…

Why do we still reach a useful W? First we visualize

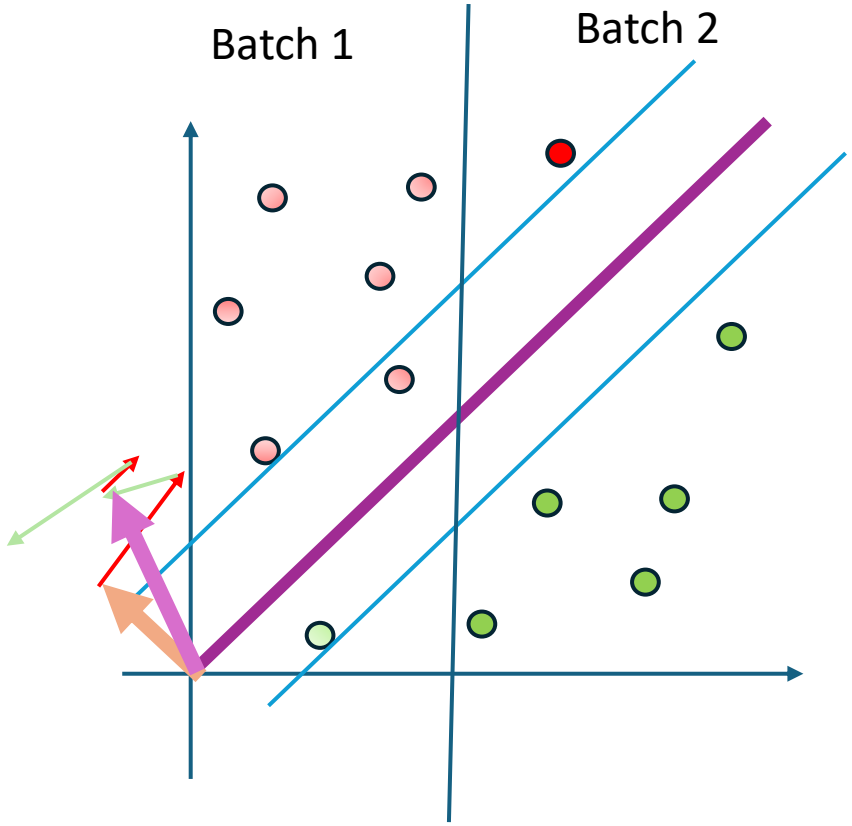
Stochastic gradient descent



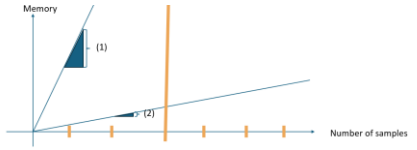
We split the data into **multiple batches** and calculate the gradient on the first batch.

Then we do a step in the direction of this gradient.

Then we calculate the gradient on the second batch.



Stochastic gradient descent

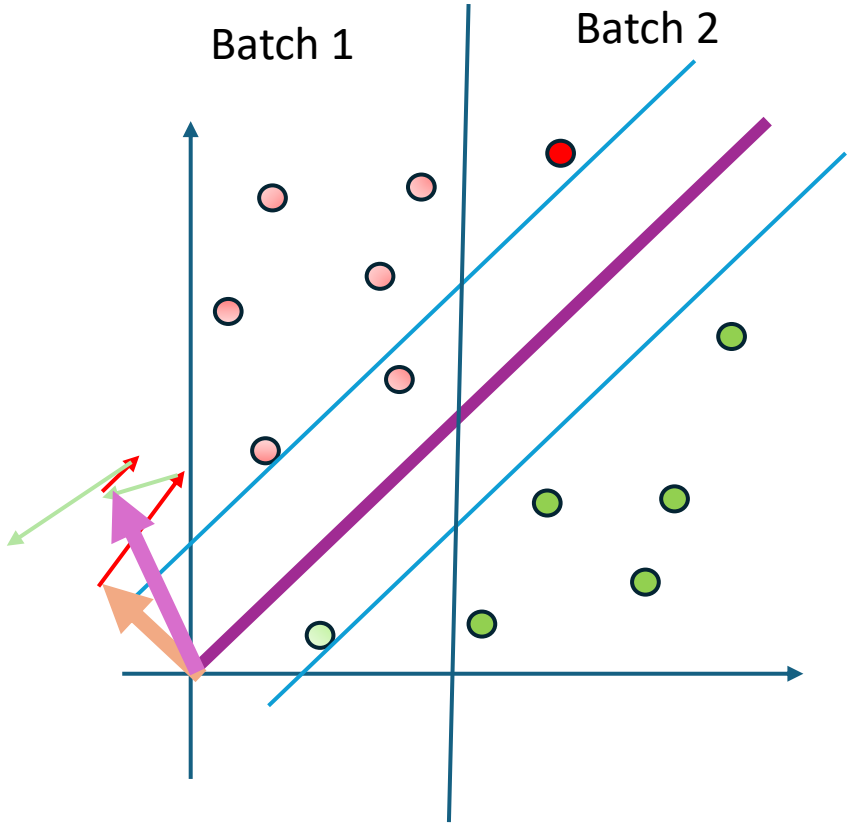


How is the gradient of the first batch related to the gradient on all training samples?

➔ Both gradients are estimates of the same vector: The Expected gradient!

$$\nabla_W E[CrossEnt\left(P\Big|_x, Q_W\Big|_X\right)]$$

➔ But the smaller the batch the bigger the variance
(which is the squared distance between the expectation and the relation we see)



$$W_0 + \mu(\nabla_W L(W_0)+\epsilon_1) + \mu(\nabla_W L(W_1)+\epsilon_2) + \cdots = W_0 + \mu[\nabla_W L(W_0) + \nabla_w L(W_1) + \cdots] + \mu[\epsilon_1 + \epsilon_2 + \cdots]$$

Contains an error, but small

$\approx E(\epsilon) = 0$

Stochastic gradient descent

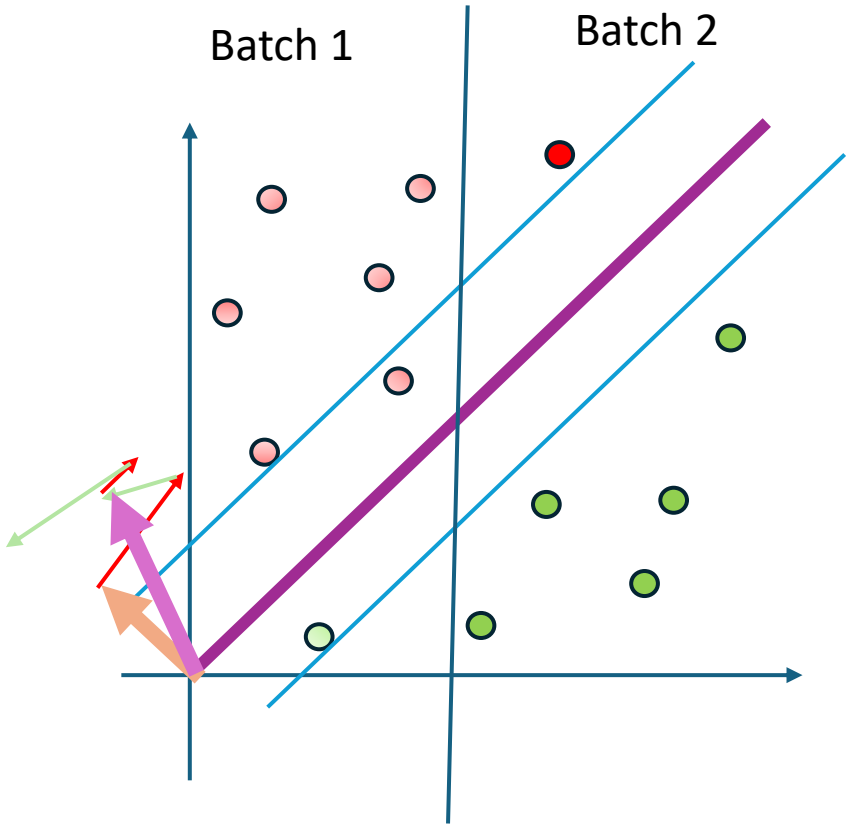
The memory we need growth linearly with the number of samples and the network dimension.

What if the memory is to small for all samples?

We split the data into multiple batches and calculate the gradient on the first batch.

Then we do a step in the direction of this gradient. Then we calculate the gradient on the second batch.

How is the gradient of the first batch related to the gradient on all training samples?



➔ Both gradients are estimates of the same vector: The Expected gradient!

$$\nabla_W E[CrossEnt(P|_x, Q_W|_X)]$$

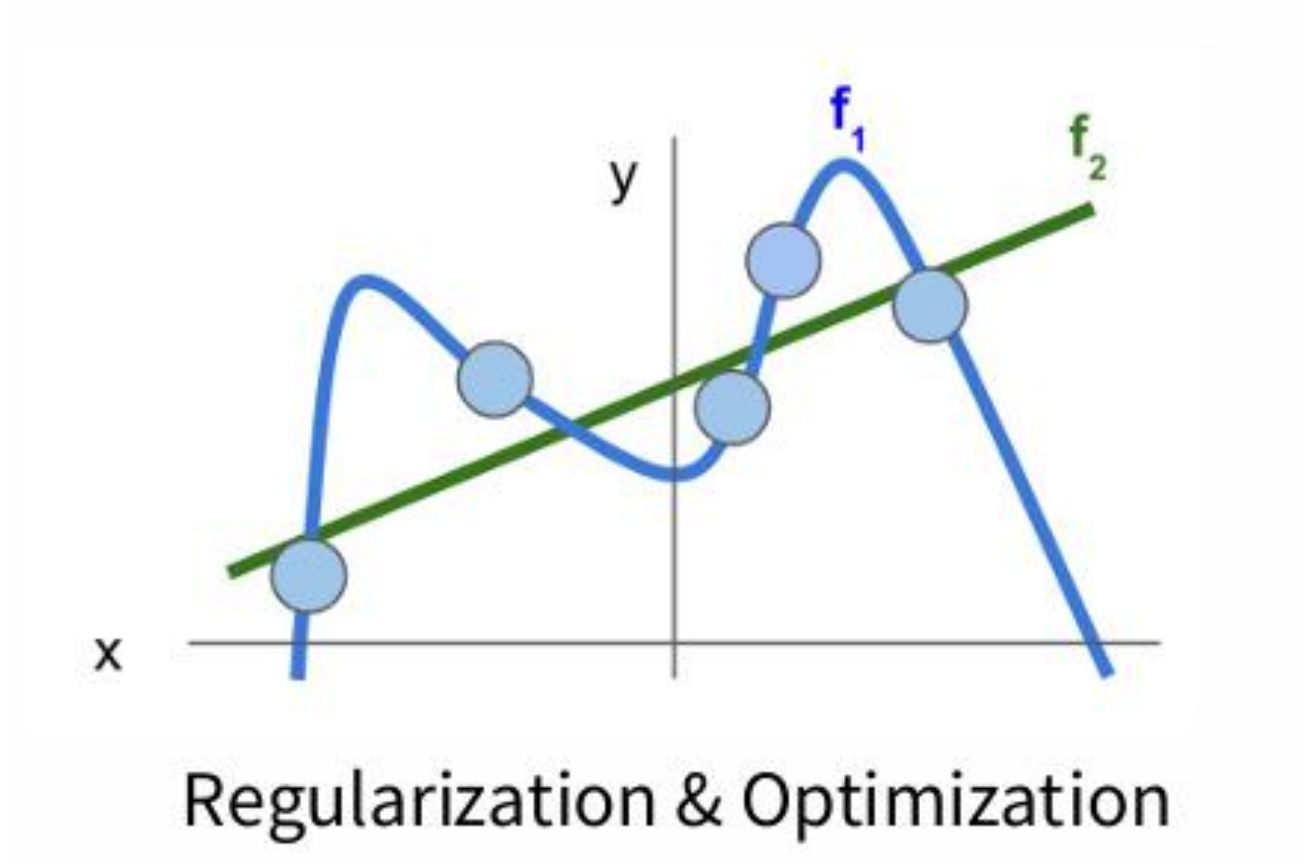
➔ The smaller the batch the bigger the variance (which is the squared distance between the expectation and the relation we see)

$$W_0 + \mu(\nabla_W L(W_0) + \epsilon_1) + \mu(\nabla_W L(W_1) + \epsilon_2) + \cdots = W_0 + \mu[\nabla_W L(W_0) + \nabla_w L(W_1) + \cdots] + \mu[\epsilon_1 + \epsilon_2 + \cdots]$$

Contains an error, but small

$\approx E(\epsilon) = 0$

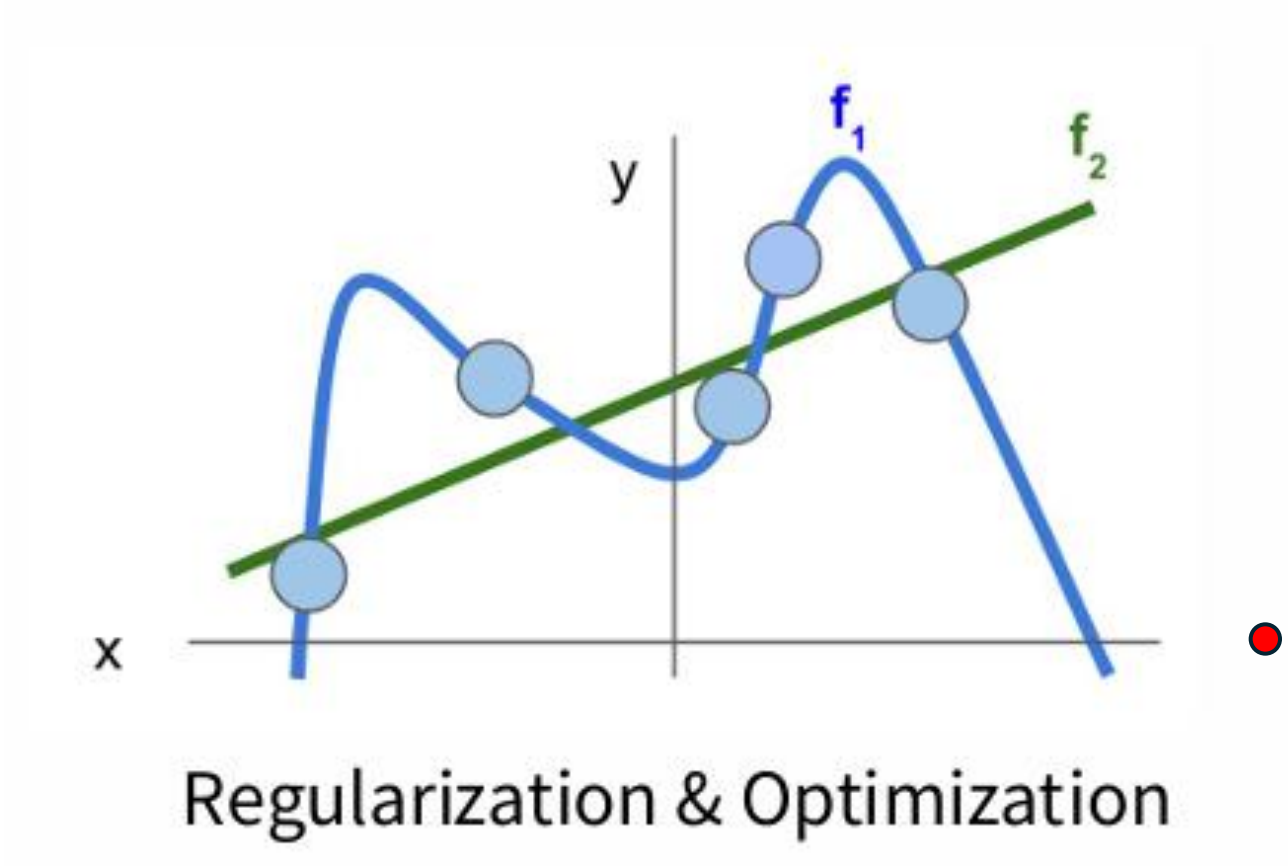
Personality test



If you do not no anything about the relation of x and y. Which model would you preferer?

A: B:

Personality test



If you do not no anything about the relation of x and y. Which model would you preferer?

A: B:

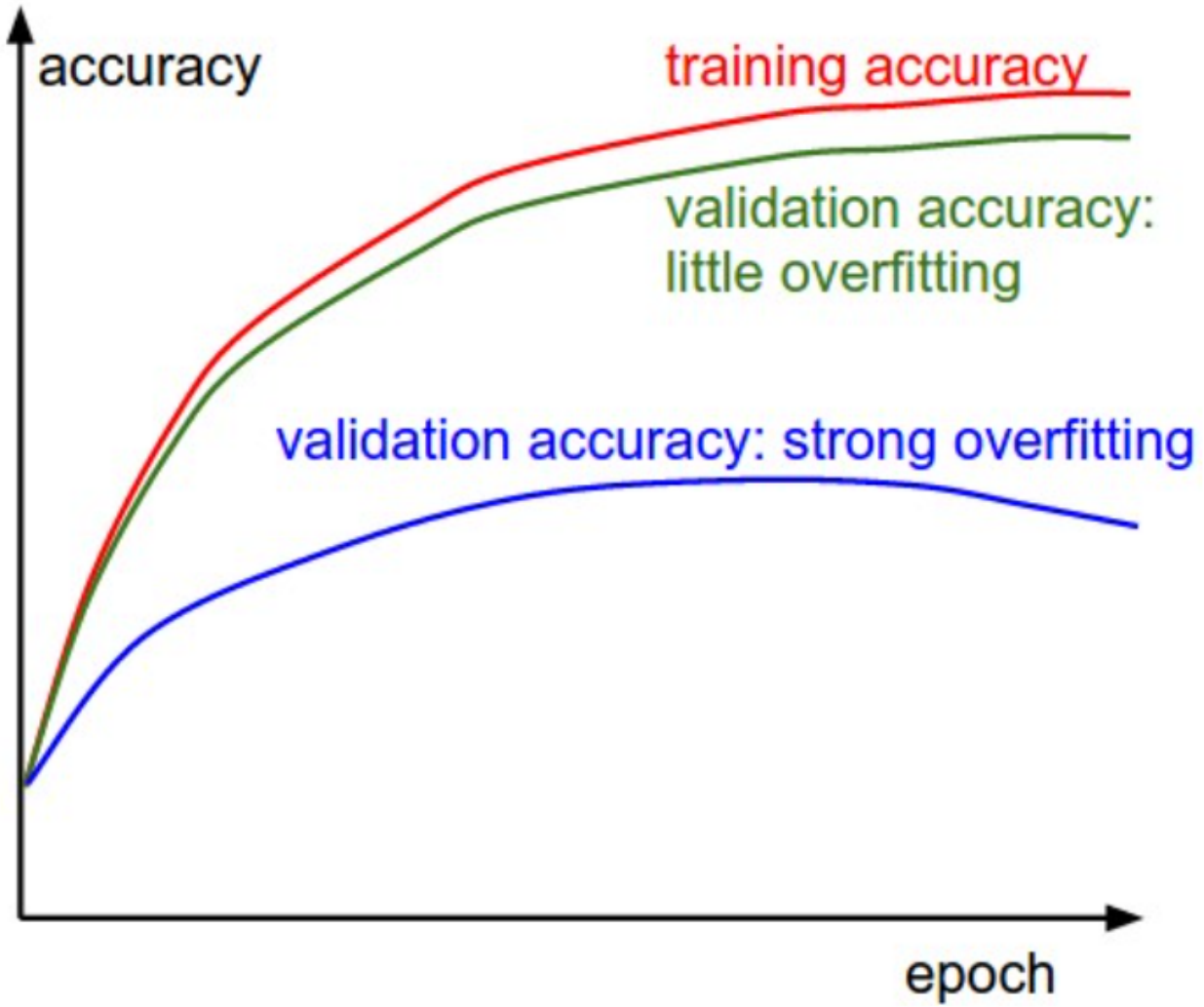
Hint which model would you use for predicting the y value at ? Why is this hint helpful?

Is a small loss all we need

No we like to make sure that we have good performance during application time.

During Training observe loss and performance measure on train & validation.

You should always look at two metrics: The loss and the quantity your interested in.



The gap between the training and validation accuracy indicates the amount of overfitting. Two possible cases are shown in the diagram on the left. The blue validation error curve shows very small validation accuracy compared to the training accuracy, indicating strong overfitting (note, it's possible for the validation accuracy to even start to go down after some point). When you see this in practice you probably want to increase regularization (stronger L2 weight penalty, more dropout, etc.) or collect more data. The other possible case is when the validation accuracy tracks the training accuracy fairly well. This case indicates that your model capacity is not high enough: make the model larger by increasing the number of parameters.

Pytorch:

input : γ (lr), θ_0 (params), $f(\theta)$ (objective), λ (weight decay),
μ (momentum), τ (dampening), *nesterov*, *maximize*

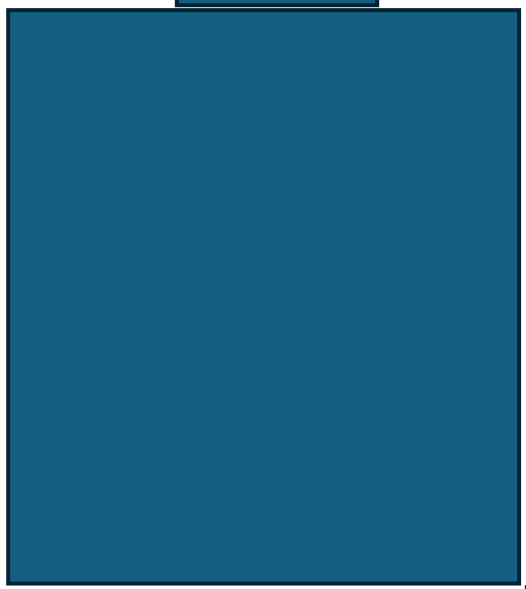
for $t = 1$ **to** ... **do**

if *maximize* :

$g_t \leftarrow$ 1

else

$g_t \leftarrow$ 



$\theta_t \leftarrow \theta_{t-1} - \gamma g_t$

Here we update the weights

return θ_t

Q: How to fill the blank 1:

A: $-\nabla_\theta f_t(\theta_{t-1})$

B: $\nabla_\theta f_t(\theta_{t-1})$

Pytorch:

input : γ (lr), θ_0 (params), $f(\theta)$ (objective), λ (weight decay),
μ (momentum), τ (dampening), *nesterov*, *maximize*

for $t = 1$ **to** ... **do**

  if *maximize* :

    $g_t \leftarrow -\nabla_\theta f_t(\theta_{t-1})$

  else

    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$

  if $\lambda \neq 0$

    $g_t \leftarrow g_t + \lambda\theta_{t-1}$

  $\theta_t \leftarrow \theta_{t-1} - \gamma g_t$

return θ_t

Effect of weight decay:
A: weight moves to zero
B: columns get orthogonal

Here we update the weights

Pytorch:

input : γ (lr), θ_0 (params), $f(\theta)$ (objective), λ (weight decay),
μ (momentum), τ (dampening), *nesterov*, *maximize*

for $t = 1$ **to** ... **do**

  if *maximize* :

    $g_t \leftarrow -\nabla_\theta f_t(\theta_{t-1})$

  else

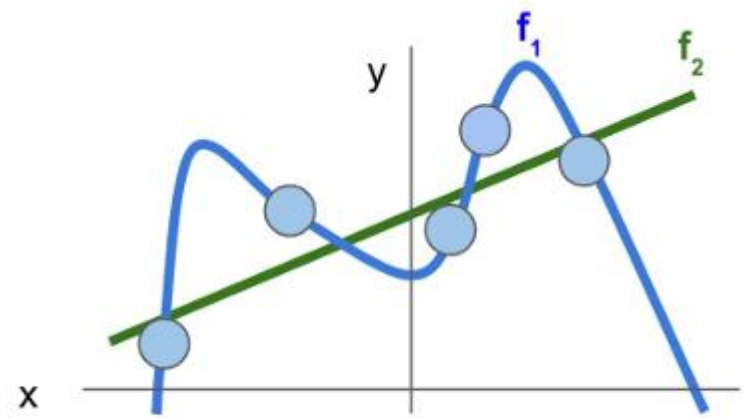
    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$

  if $\lambda \neq 0$

    $g_t \leftarrow g_t + \lambda\theta_{t-1}$

  $\theta_t \leftarrow \theta_{t-1} - \gamma g_t$ ← Here we update the weights

return θ_t



Regularization & Optimization

The higher we choose the weight decay the more likely is

A: B:

Pytorch:

input : γ (lr), θ_0 (params), $f(\theta)$ (objective), λ (weight decay),
μ (momentum), τ (dampening), *nesterov*, *maximize*

for $t = 1$ **to** ... **do**
  if *maximize* :
    $g_t \leftarrow -\nabla_\theta f_t(\theta_{t-1})$
  else
    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$
  if $\lambda \neq 0$
    $g_t \leftarrow g_t + \lambda\theta_{t-1}$
  if $\mu \neq 0$
    if $t > 1$
      $\mathbf{b}_t \leftarrow \mu\mathbf{b}_{t-1} + (1-\tau)g_t$
    else
      $\mathbf{b}_t \leftarrow g_t$
  $g_t \leftarrow \mathbf{b}_t$
  $\theta_t \leftarrow \theta_{t-1} - \gamma g_t$

return θ_t

Momentum …
A: … helps to reduce noise from the stochastic gradients
B: … steers the NN to narrow optima.

Here we update the weights